Received: 6 July 2021 | Revised: 22 November 2021 | Accepted: 29 November 2021

DOI: 10.1002/stvr.1804

SURVEY ARTICLE

WILEY

Improving test automation maturity: A multivocal literature review

Yuqing Wang | **Mika V. Mäntylä** | **Zihao Liu** | **Jouni Markkula** | **Päivi Raulamo-jurvanen**

M3S research unit, University of Oulu, Pentti Kaiteran katu 1, Oulu, Finland

Correspondence
Yuqing Wang, M3S research unit, University of Oulu, Pentti Kaiteran katu 1, Oulu 90014, Finland.
Email: yuqing.wang@oulu.fi

Funding information
ITEA3; Tauno Tönningin Säätiö; Business Finland, Grant/Award Number: 3192/31/2017; Tauno Tönning, Grant/Award Number: 20210086

Abstract
Mature test automation is key for achieving software quality at speed. In this paper, we present a multivocal literature review with the objective to survey and synthesize the guidelines given in the literature for improving test automation maturity. We selected and reviewed 81 primary studies, consisting of 26 academic literature and 55 grey literature sources. From primary studies, we extracted 26 test automation best practices (e.g., Define an effective test automation strategy, Set up good test environments, and Develop high-quality test scripts) and collected many pieces of advice (e.g., in forms of implementation/improvement approaches, technical techniques, concepts, and experience-based heuristics) on how to conduct these best practices. We made main observations: (1) There are only six best practices whose positive effect on maturity improvement have been evaluated by academic studies using formal empirical methods; (2) several technical related best practices in this MLR were not presented in test maturity models; (3) some best practices can be linked to success factors and maturity impediments proposed by other scholars; (4) most pieces of advice on how to conduct proposed best practices were identified from experience studies and their effectiveness need to be further evaluated with cross-site empirical evidence using formal empirical methods; (5) in the literature, some advice on how to conduct certain best practices are conflicting, and some advice on how to conduct certain best practices still need further qualitative analysis.

KEYWORDS
improvement, maturity, practice, software, systematic literature review, test automation

1 | INTRODUCTION

Modern software development approaches, such as Agile, DevOps and continuous integration, have changed the way software is being developed [1,2]. Software products are released faster and more frequently than before—software release frequency improved from around 12 months in the last century to 3 weeks these days [2]. How to ensure development speed without sacrificing quality is essential. Quality issues may lead to disappointments of customers, reputation damage of software products and security problems—thus, the loss of the market [3]. Test automation is key for achieving 'software quality at speed' in modern software developments, because of the rapid speed it offers to test software products in short test cycles [1,4]. With the rise of modern software developments, test automation is growing in popularity. According to the statistics [5], 'the global test automation market size is expected to grow from USD 12.6 billion in 2019 to USD 28.8 billion by 2024'.

However, many organizations still have immature test automation practices with negative outcomes, for example, unable to automate valuable tests, create and maintain automated tests at optimal costs, or detect defects

This is an open access article under the terms of the Creative Commons Attribution License, which permits use, distribution and reproduction in any medium, provided the original work is properly cited.
© 2022 The Authors. *Software Testing, Verification & Reliability* published by John Wiley & Sons Ltd.

on time. Such negative outcomes impede organizations to reap the expected test automation benefits, consume software development resources, and even risk software quality. The software industry and research community refer to 'improving immature test automation practices with negative outcomes' as 'improving test automation maturity' [6–8]. The software industry has seen increasing attempts to improve test automation maturity but not all attempts are effective [9,10]. Based on ISTQB's recent software testing practice survey [9], around 65% of the almost 2000 software organizations in the world attempted to improve test automation maturity while only about a half reported that their attempts are effective. The industry needs the guidelines to drive for effective test automation maturity improvement [9–11]

The current literature includes many guidelines for improving test automation maturity [4,12,13]. Surveying and synthesizing the guidelines given in the literature is essential to integrate and link the guidelines from different sources into a whole. Yet the research effort on that is limited [4,14]. Our previous work [14] stands for the purpose to narrow this gap. Our previous work is the first attempt to survey the existing guidelines and synthesize test automation best practices. It reviewed 18 test maturity models that are used to guide test automation practices in the industry. From these 18 test maturity models, our previous work extracted some test automation best practices but failed to identify qualitative analysis on these best practices and how they can be conducted. This calls for a systematic literature review (SLR) on a larger pool of sources to synthesize test automation best practices with further qualitative analysis.

This paper presents a multivocal literature review (MLR) with the objective to survey and synthesize the guidelines given in the literature for improving test automation maturity. It intends to complement the current research that lacks an SLR of a large pool of sources to synthesize test automation best practices with further qualitative analysis. MLR is a type of SLR that includes both academic literature (AL) and grey literature (GL). Our reason to use the MLR approach is the industrial relevance of our topic. Based on prior scholars [15,16], GL (e.g., white papers, magazines, blog posts, wikis, and technical reports) is 'still an unexplored gold mine of guidelines for test automation', as practitioners exist in large numbers and they are sharing their experience matured on the field and propose test automation guidelines. Surveying the guidelines for improving test automation maturity from both AL and GL is critical to enable knowledge transfer between industry and research and identify the gaps.

In this MLR, we searched the AL using Google Scholar, Scopus, and Web of Science, and the GL using the Google search engine on the topic of test automation maturity improvement. From a large pool of sources, we selected 81 primary studies (consist of 26 AL and 55 GL sources) and studied them. As the main contributions, from these 81 primary studies, we extracted 26 test automation best practices using the thematic analysis approach, and collected many pieces of advice (in forms of implementation/improvement approaches, actions, technical techniques, concepts and experience-based heuristics) on how to conduct these best practices. Using cited sources, we explained these 26 best practices and the advice around them. We have several observations related to these 26 best practices and the advice around them:

- There are only six best practices (define an effective test automation strategy, provide enough resources, have competent test professionals, select the right test tools, set up good test environments, design the SUT for automated testability) whose positive effect on maturity improvement have been evaluated by academic studies using formal empirical methods.
- This MLR identified six technical related best practices (set up good test environment; create high-quality test data; develop high-quality test scripts; automate test oracles; analyse test automation results efficiently and effectively; adopt new technologies), which were not presented in test maturity models reviewed in our previous work [14].
- Some best practices identified in this MLR can be linked to test automation success factors and maturity impediments proposed by other scholars.
- Most pieces of advice on how to conduct proposed best practices were identified from experience studies and their effectiveness need to be further evaluated with cross-site empirical evidence using formal empirical methods.
- In the literature, some advice on how to conduct several best practices are conflicting, for example, as the advice for 'Select the right test tools', many scholars suggested selecting test tools against pre-defined selection criteria, while some practitioners viewed that - based on their experience—selecting test tools against pre-defined criteria is less useful than selecting each test tool with an experimentation mindset.
- In the literature, some advice on how to conduct certain best practices still need further qualitative analysis, for example, as the advice for 'Use the right test automation metrics', the literature defined the concept of 'right test automation metrics' and illustrated a collection of example ones, but it did not mention how to define and customize test automation metrics based on own needs of organizations.

Our study provides a single source that surveyed and synthesized the guidelines given in both AL and GL for test automation maturity improvement. It can be useful to both practitioners and researchers, who can, respectively, consult

our 26 test automation best practices with the advice on conducting them for improving test automation maturity, and foster future research in this field.

The remainder of this paper is structured as follows. Section 2 introduces the concepts, explains MLR approach, and reviews related work. Section 3 describes the research process and methodology used to conduct this MLR. Section 4 reports the study results. Section 5 summarizes and discusses the study findings, explores the implications of this MLR to research and practice, and examines threats to validity. Section 6 concludes the paper and states its contributions.

2 | BACKGROUND

In the following subsections, we study the general concept of maturity and software test automation maturity in Section 2.1, introduce the MLR approach in software engineering (SE) in Section 2.2, and review related work in Section 2.3.

2.1 | The concept of maturity and software test automation maturity

According to the Oxford dictionary [17], maturity is defined as: 'the state of being fully grown or developed'. The concept of maturity has been introduced into various fields. For instance, software process maturity in SE [18], business process maturity in the area of business management [19], and big data maturity in data science. In a literature review study [20], Mettler has studied the concept of maturity in different fields and concluded three forms of maturity:

- Process maturity, a specific process 'is explicitly defined, managed, measured, controlled, and effective'.
- Object maturity, a particular object 'like a software product, a machine or similar reaches a pre-defined level of sophistication'.
- People capability, the work force 'is able to enable knowledge creation and enhance proficiency'.

Based on test automation literature and Mettler's views of maturity, test automation maturity consists of three dimensions; each dimension is dependent on the other. Thus, this MLR considers test automation maturity improvement with those three dimensions as the whole:

- Test automation process maturity: *a test automation process is explicitly defined, managed, measured, controlled, and effective* [7,12,21]. A test automation process is a sequence of test automation activities [12,21]. Test automation process maturity demonstrated that all test automation activities are organized in a structured process to produce expected outcomes [12,21,22].
- Test automation technology maturity: *test automation technology reaches a pre-defined level of sophistication to meet the needs of software development* [13,22,23]. Test automation technology is a set of techniques, expertise, tools, methods and processes that can be used in the development and application of test automation [22]. Test automation is technology-driven [13]. The increased level of sophistication of test automation technology allows the increased test scale, test coverage, test efficiency and test effectiveness [23].
- People capability: *the workforce is able to leverage the competencies in test automation* [4,24,25]. A variety of expertise is required for people to be able to work on test automation [4,25]. People capability concentrates on continuously improving the management and development of human assets to ensure that the right people with enough expertise are playing the roles in test automation [24,25].

2.2 | Multivocal literature review

SLR is a popular research methodology to conduct literature reviews for aggregating all existing evidence on research questions SE [26]. MLR is a type of SLR that includes both AL and GL sources (e.g., white papers, web pages, magazines, technical reports, wikis and blog posts) [16]. Recently, MLR has been actively applied in SE research and is attracting attention [27]. Prior MLRs have made contributions to SE research. For instance, MLR [4] contributed five groups of factors to support automation decisions in software testing; MLR [28] identified 58 test improvement models and contributed the drivers, challenges and benefits of using these models; MLR [29] contributed the concept, characteristics, benefits and challenges of DevSecOps.

As concluded by Garousi et al. [16], SE research needs MLRs for many reasons. First, GL is the source of up-to-date research, as SE practitioners are actively producing fresh GL sources on a large scale. Second, neglecting GL may not able to provide enough insights into 'the state of the art and practices' in the current SE context. Third, SE research is practical-oriented; thus, it is essential to hear the voice of both SE researchers and practitioners in order to narrow the gap between academia and industry. In this paper, our reason to use the MLR approach relates to the industrial relevance of our topic and is consistent with prior SE researchers who conduct MLRs.

2.3 | Related work

Some secondary studies have surveyed the literature on test automation maturity improvement. A summary of these studies is in Table 1. In Table 1, the main contributions of each study are presented in column 'Contributions'. By

TABLE 1 Relevant secondary studies

Ref.	Method	Primary study	Contributions	Gaps	Our MLR
[30]	SLR and Survey	25 AL sources	This study found that maximizing the benefits (improve product quality, test coverage, reliability and fault detection) and addressing drawbacks (lack of skilled people and test automation needs time to be mature) can improve test automation maturity.	A taxonomy of benefits and drawbacks is too high level. How to maximize proposed benefits and address proposed drawbacks in practice was not mentioned.	Our MLR proposed 26 test automation best practices (grouped into 13 key areas) that are more practical. It also collected the advice of prior researchers and practitioners about how to conduct each best practice.
[13]	SLR	39 AL sources	Five categories of impediments that hinder test automation maturity: behavioural effects, business and planning, skills, test systems, system under test.	Five categories of impediments are too high level and miss some test automation key areas (test design, test execution, verdicts and measurements). How to tackle those implements for improvements were barely mentioned.	Our MLR proposed 26 test automation best practices (grouped into 13 key areas) that are more practical. Our MLR concerned test automation key areas (test design, test execution, verdicts and measurements) that were not covered in their study. Best practices in our MLR can be used to tackle the relevant impediments in their work.
[31]	LR and survey	4 AL sources	12 critical success factors of test automation: feasibility assessment, testability level of the software under test, resource availability, manageability, well-defined test process, scalability, maintainability, automation tool acquisition criteria, quality control, resource reusability, dedicated and skilled team, and automation planning/ strategy.	Not a SLR. Their study missed some test automation key areas (test environment, test automation requirements, test design, test execution and verdicts). How to improve test automation maturity against those critical success factors was not introduced.	Our MLR is an SLR. Our MLR concerned test automation key areas (test environment, test automation requirements, test design, test execution and verdicts) that were not covered in their study. Best practices in our work can be used to achieve relevant success factors in their work.
[4]	SLR	26 AL + 52 GL sources	Five types of factors affect when/what to automate in software testing decisions: software under test-related factors, test-related factors, test-tool-related factors, human and organizational factors, cross-cutting and other factors.	This study only addresses what and when to automate decisions; it did not advise the improvement steps after initial decisions in a test automation process.	Our MLR is an SLR. Our work advises best practices throughout the whole lifecycle of a test automation process.

(Continues)

TABLE 1 (Continued)

Ref.	Method	Primary study	Contributions	Gaps	Our MLR
[14]	LR	18 test maturity models and expert reviews	Test automation best practices in 13 key areas: test automation strategy, resources, test organization, knowledge transfer, test tools, test environment, test automation requirements, test design, test execution, verdicts, test automation process, measurements, SUT.	Not a SLR. It only named test automation best practices. Why each best practice is suggested and how it can be conducted were not included.	Our MLR proposed test automation best practices with an SLR. We identified several technical related best practices that did not present in their work. We explained the proposed best practices and collected many pieces of advice on how to conduct proposed best practices.

further investigation, we found that the contributions of each of these studies are insufficient to guide the industry to improve test automation maturity, the specific reasons for each study are listed in column 'Gaps'. How our MLR narrows the gap of the prior one is concluded in column 'Our MLR'.

To sum up, our MLR complements prior studies and makes the novelty from the following aspects:

- This MLR concerned 13 test automation key areas to study test automation maturity improvement. Compared with most previous secondary studies, this MLR concerned more test automation key areas.
- Our MLR reviewed both AL and GL sources on test automation maturity improvement. Due to the industrial relevance of this research topic, including the GL is necessary. Only one prior secondary study [4] included GL and that study was about what and when to automate in software testing.
- This MLR proposed test automation best practices that allow for improving test automation maturity. Our previous work [14] is the only one that also proposed test automation best practices. This MLR identified several technical related best practices that were not presented in our previous work, see details in Section 5.1. This MLR complements our previous work by exploring why each best practice is suggested and collecting the advice of prior researchers and practitioners on how to conduct proposed best practices.
- This MLR was recently finished and intended to provide the current view for this research scope. Prior secondary studies were published in 2016 or earlier years.

3 | RESEARCH METHOD

Our MLR was carried out following 'the guidelines for conducting MLRs' from Garousi et al. [16]. Our research process contains six stages: review question, search strategy, source selection process, quality assessment, mapping of primary studies, and data extraction and synthesis. Each stage is described below.

3.1 | Review question

To address the objective of this MLR, we defined two review questions:

- **RQ1. Which test automation best practices are given in the literature?** This review question aims to extract a taxonomy of test automation best practices from the literature.
- **RQ2. What advice was given in the literature about how to conduct proposed test automation best practices?** This review question attempts to explore the advice in the literature about how to conduct proposed test automation best practices in RQ1.

3.2 | Search strategy

The search strategy was developed to search for relevant AL and GL sources to answer our review questions in this MLR. The first author developed the search strategy and it was reviewed and revised by other co-authors. The search strategy defined the search string, search databases, inclusion and exclusion criteria and the source selection process.

TABLE 2 Search string

Row	Terms
A	software
B	test OR testing
C	automation OR automated
D	mature OR maturity OR maturation OR matureness
E	improvement OR improve OR assessment OR assess

TABLE 3 Inclusion criteria

C1. The AL or GL source is written in English.
C2. The AL or GL source is full-text accessible.
C3. The AL or GL source is not a duplication of others.
C4. The AL source was published in journals, conferences, workshops, and books.
C5. The GL source was published in blogs, magazines, web pages, white papers, technical reports, tutorials, wikis, presentations, broadcastings, videos.
C6. The AL or GL source is relevant to test automation maturity improvement

3.2.1 | Search string

Our search string was defined by following the standard steps proposed by Brereton et al. [32]. Five major terms were defined according to our research topic: software, test, automation, maturity and improvement. Alternative synonyms of those major terms were identified by reviewing prior literature. The search string was then defined accordingly, see Table 2. The complete search string is formulated by combining the rows with AND.

3.2.2 | Search databases

We conducted a preliminary search in different bibliographic databases and search engines. Search databases were selected depending on the quality and availability of relevant sources on them. We decided to use Scopus, Web of Science and Google Scholar to search for AL sources. With our search string, Scopus and Web of Science can return many quality AL sources relating to our research topic. Google Scholar allows retrieving the full-text of AL sources from open access journals and pre-print repositories. The Google search engine was chosen to search for GL sources. It can return more relevant GL sources with our search string compared to other search engines. However, prior researchers have argued that using search engines may introduce the selection bias into SLRs, because the ranking algorithm of search engines may tailor search results based on user preferences and behaviours [33]. Studies [34,35] have introduced the approaches to reduce such selection bias in SLRs. In this MLR, we took the following approaches:

- The search in Google Scholar or Google search engine was done without logging into a Google account. The search history and cache in the browser were cleared up before the search.
- After applying the search string in Google Scholar or Google search engine, search results were exported into a spreadsheet that stores the title and URL of each source. In the source selection process, we open each URL to screen the sources.

3.2.3 | Inclusion and exclusion criteria

Table 3 shows inclusion criteria to select relevant sources for this MLR. Exclusion criteria are the reverse of inclusion criteria. This means a source that does not meet inclusion criteria will be excluded.

3.3 | Source selection process

The search selection process consists of four phases, see Figure 1. Table 4 summarizes the search results of different phases. In Phase 1, our search string was applied in the chosen search databases. It was modified to fit the search format of each database. The search results were shown in each database.

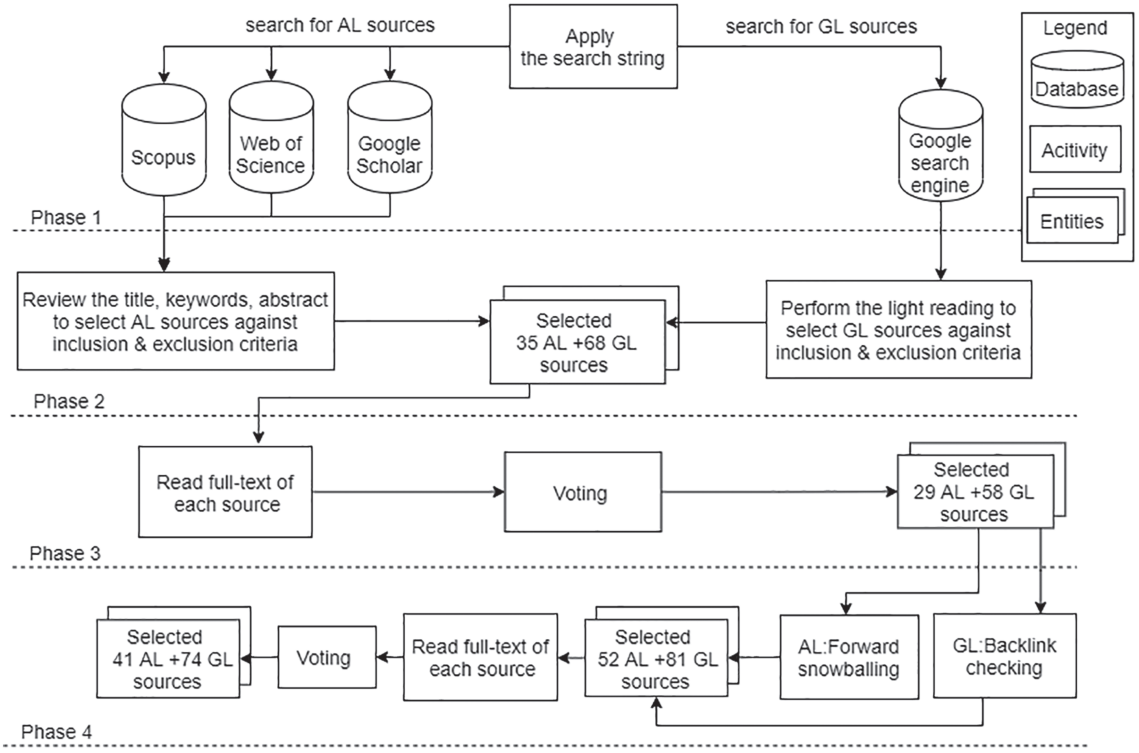


FIGURE 1 An overview of search selection process

TABLE 4 Summary of search results

Databases	Phase 1	Phase 2	Phase 3	Phase 4
Scopus	76	5	2	3
Web of Science	32	2	1	1
Google Scholar	354,000+	28	26	37
Google search engine	32,500,000+	68	58	74

In Phase 2, sources appearing in search results of Phase 1 were reviewed against the defined inclusion and exclusion criteria (Section 3.2.3). The title, abstract and keywords were reviewed to screen AL sources. Light reading was performed to screen GL sources. Note that, though there were more than 354,000 search results in Google Scholar and 32,500,000 search results in Google search engine, the relevant sources only presented in the first few pages. Thereby, in Google Scholar and Google search engine, sources on first-10 pages were screened. At the end of this phase, 35 AL and 68 GL sources were selected for further review.

In Phase 3, the first and third authors read full-text of each source selected from Phase 2. They used 'yes' or 'no' to vote for whether a source meets each inclusion criterion. Only the source got 'yes' for all inclusion criteria from each author were selected. In the end, 29 AL and 58 GL sources were selected to build up our initial pool of sources.

In Phase 4, we used the Forward snowballing approach to screen additional AL sources based on the citations of selected AL sources in Phase 3, and checked the backlinks of selected GL sources in Phase 3 to screen additional GL sources. This screened additional 23 AL and 23 GL sources. The first and third authors read full-text of each of these sources against the defined inclusion and exclusion criteria (Section 3.2.3). They used 'yes' or 'no' to vote for whether a source meets each inclusion criterion. Only 12 AL and 16 GL sources got 'yes' for all inclusion criteria from each author were selected. Our final pool consists of 41 AL and 74 GL sources.

3.4 | Quality assessment

The quality of sources in our final pool was assessed to minimize bias and maximize the validity of this MLR. Since AL and GL sources do not follow the same review and publication process [36], a separate checklist was created to assess the quality of each type of sources.

To assess the quality of AL sources, we studied quality assessment guidelines from Zhou et al. [37] and Kitchenham [38]. We developed a checklist that contains 20 quality-check questions for assessing the quality of AL sources in this MLR. Besides, Garousi et al.'s GL quality assessment checklist [16] was used to assess the quality of GL sources in this MLR. We revised their checklist to fit our needs. Our checklist for GL sources contains 17 quality-check questions. Our checklists for both AL and GL sources are presented online (https://doi.org/10.6084/m9.figshare.13554164.v1).

To rate the quality of each source, we used 3-point-scale (1 = yes, 0.5 = partly, 0 = no) to answer quality-check questions one by one in each of our checklists. This method has been used in prior studies, for example, the MLR studies from Garousi and Mäntylä [4] and Garousi [28]. The total score of each source was calculated. Sources scored 50% or more of the maximum points were finally included. This means, to be included, the AL source should get at least 10 out of 20 points, while the GL source should get 8.5 or more out of 17 points. As a result of quality assessments, 15 AL and 19 GL sources were excluded. The rest 81 sources (including 26 AL and 55 GL sources) were used as primary studies for this MLR. For the sake of transparency, we listed out all 81 primary studies in Appendix A: P1–P26 are AL sources while P27–P81 are GL sources. Many highly cited SLR guidelines [16,39,40]) recommended listing out primary studies in the appendix—that could let readers easily distinguish primary studies from other cited sources in a research paper. Many previous SLRs [4,13,28]) have followed such guidelines to do so. Besides, our paper repository can be found online (https://doi.org/10.6084/m9.figshare.17059925.v1).

3.5 | Mapping of primary studies in our final pool

We mapped our 81 primary studies in four aspects: number of sources per year, numbers of sources by source type, author profile, and number of sources by contribution and research type.

3.5.1 | Number of sources per year

Figure 2 shows the number of sources per year by literature type. The identified sources were produced between 1994 and 2021. Before the year 2012, there were less than three sources per year for both GL and AL sources. More sources were identified in a recent decade.

3.5.2 | Numbers of sources by source type

Figure 3 shows the number of sources by source type in both categories: AL versus GL. As noted in this figure, conference papers ($n = 10$) and journal papers ($n = 10$) take a large proportion of the AL sources. Most GL sources are online articles ($n = 34$) published in blogs, technical websites and magazines.

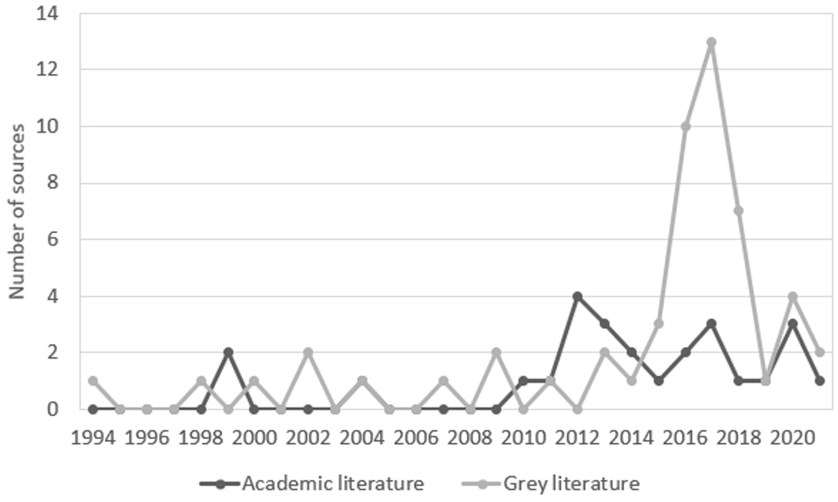


FIGURE 2 Number of sources per year

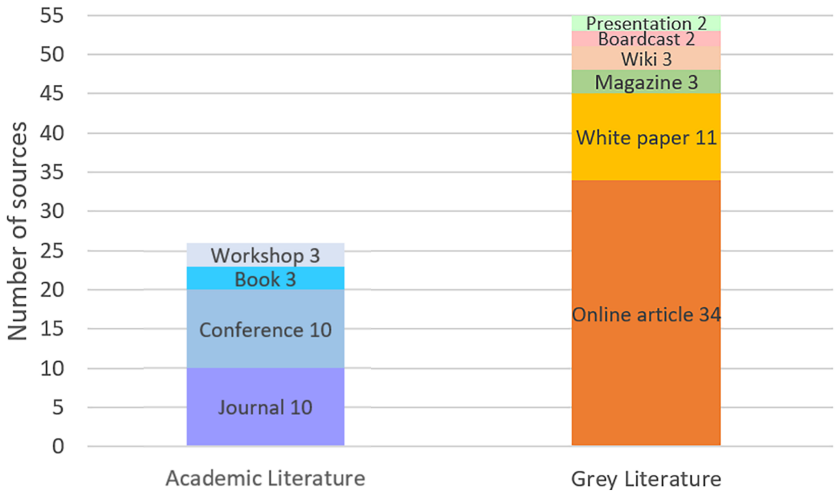


FIGURE 3 Number of sources by type

TABLE 5 Author profile of grey literature sources

	N
Identifiable individual practitioner:	
Test lead/manager/director	11
Tester	6
Developer	2
Consultant	15
Others	5
Identifiable organization	13
Unsigned author	3

3.5.3 | Author profile

All 26 AL sources have at least two authors. 12 AL sources have the author, who specializes in software testing and has more than a hundred publications. Most of those senior researchers move their interest to software test automation in recent years. Table 5 depicts the author profile of 55 GL sources in this MLR. As shown in this table, the majority of GL sources ($n = 39$) were produced by identifiable individual practitioners.

3.5.4 | Number of sources by contribution and research type

To analyse contribution types of our primary studies, we used the categories from Garousi et al. [16]: heuristics and guidelines, model, process, method, metric and tool. To analyse the research type of our primary studies, we studied the guidelines from Garousi et al. [16] and Petersen et al. [41], and accordingly classified our primary studies into four categories:

- Evaluation (Eva): These studies use formal empirical methods (e.g., practitioner survey, case studies, controlled experiments and hypothesis testing) to evaluate test automation maturity improvement related findings.
- Experience (Exp): These studies present test automation maturity improvement related findings based upon the authors' (can be individuals or software organizations) experience and expertise.
- Opinion (Opi): These studies present test automation maturity improvement related findings based on authors' (can be individuals or software organizations) opinions.
- Other: These studies do not fit into any of the above research types, including philosophical studies and survey studies.

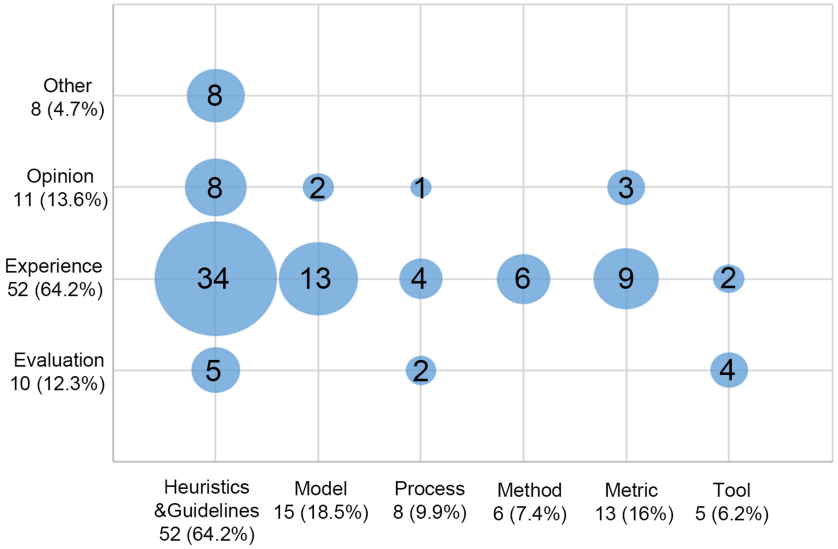


FIGURE 4 Mapping primary studies by contribution and research type

Figure 4 shows the mapping of our primary studies by contribution and research type. Note that, since a source can have multiple types of contributions, the total number of studies ($n = 99$) in all contribution types is greater than the total number of primary studies ($n = 81$). Only 10 primary studies are evaluation studies, while 4 of them purely focus on test tool selection and usage related topics. Many of our primary studies are experience studies ($n = 34$) that contributed to test automation heuristics and guidelines. 15 out of our primary studies proposed test maturity models. Four test maturity models were proposed by authorized organizations: TMap (proposed by Sogeti), TPI (proposed by Sogeti), STBox 3.0 (proposed by CTG Europe), TestSPICE 3.0 (proposed by Intacs). The rest models were proposed by individual practitioners or software organizations based on experience and expertise.

3.6 | Data extraction and synthesis

We coded our primary studies using an integrated deductive and inductive approach proposed by Cruzes and Dyba [42]. A data analysis software NVivo[1] was used to assist the coding process.

We read the full content of each primary study to identify the findings that propose test automation best practices. Relevant findings were coded from each primary study. Our prior work [14] reviewed 18 test maturity models and identified 13 test automation key areas. In this MLR, pre-defined code categories (to classify test automation best practices) were created according to these 13 test automation key areas identified in our prior work: test automation strategy, resources, test organization, knowledge transfer, test tools, test environment, test automation requirements, test design, test execution, verdicts, test automation process, measurements, and software under test (SUT). Pre-defined code categories were allowed to be modified depending on actual coding situations. All codes at this stage were assigned into at least one code category. Figure 5 is an NVivo screenshot showing coding status at this stage. In this screenshot, 'Name' represents code categories; 'Files' counts the number of cited sources; 'References' counts the total number of codes on all cited sources. The total number of files is less than the sum of references, as a single source may have many codes.

All codes were reviewed by reading through the content they included. The content of each code was displayed and examined in the original context of a cited source, considering the surrounding texts of the code. An annotation was written to conclude a concise description of each code. Based on codes and annotations, we created high-level categories upon initial code categories using an inductive approach. Each high-level code category represented a test automation best practice. All codes were assigned into at least one high-level category. Figure 6 shows an NVivo screenshot that gives examples: high-level categories 'select the right test tools' and 'test tool usage' were created under the initial code category 'test tools'; example codes were assigned to the high-level category 'select the right test tools'; An annotation concluded a code.

Based on codes in high-level coding categories, a taxonomy of test automation best practices was formulated. We refined best practices based upon the content of codes, annotations and cited sources until a final taxonomy was

[1]NVivo: https://www.qsrinternational.com/nvivo-qualitative-data-analysis-software/

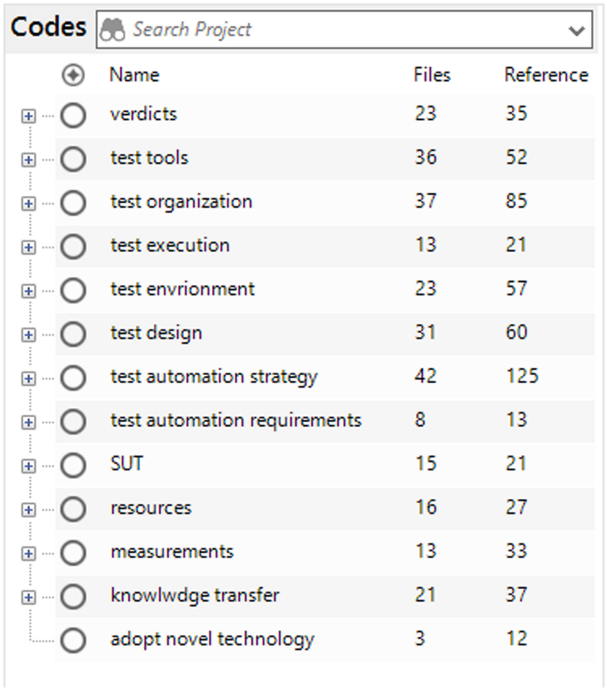


FIGURE 5 NVivo screenshot 1: coding status at the initial phase

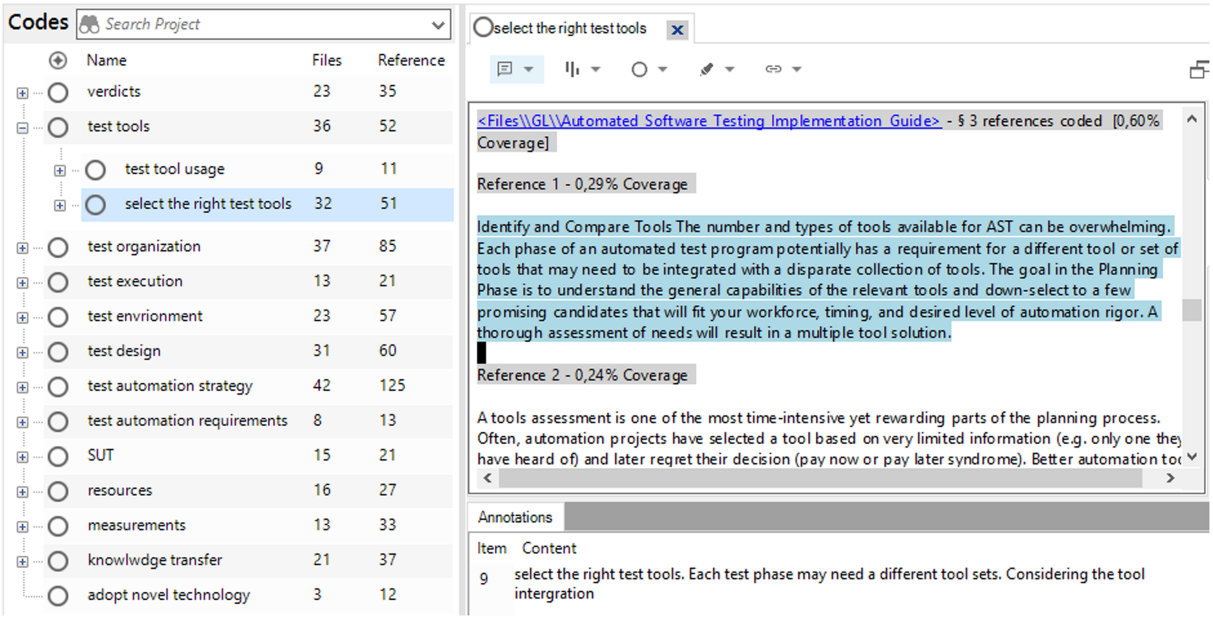


FIGURE 6 NVivo screenshot 2: examples of high-level code categories, codes and annotations

obtained. By reviewing the content of codes, annotations and cited sources again, we collected the advice on how to conduct these best practices. To answer our RQ1, we intended to present our final taxonomy of test automation best practices and demonstrate cited sources by source type (AL and GL) and research type (Eva, Exp, Opi and Other). To answer our RQ2, using cited sources, we intended to explain each best practice and present the advice on how to conduct it. The explanation and advice to best practices were distinguished by source type (AL and GL) and research type (Eva, Exp, Opi and Other).

4 | RESULTS

In the following subsections, we present the study results to our review questions in turn.

4.1 | RQ1—Test automation best practices

We extracted 26 test automation best practices from our 81 primary studies (that consist of 26 AL and 55 GL sources). Table 6 shows these 26 best practices grouped into 13 key areas. We had two observations on Table 6. First, we found that, some best practices ('Adjust the strategy to the changes', 'Automate test oracles' and 'Adopt new technologies') suggested in the GL did not be suggested in the AL from our primary studies. Second, we noticed that, there are only six best practices ('Define an effective test automaton strategy','Provide enough resources', 'Have competent test professionals', 'Select the right test tools', 'Set up good test environments' and 'Design the SUT for automated testability') whose positive effect on test automation maturity have been validated by academic evaluation studies (from our primary studies) using formal empirical methods.

TABLE 6 Best practices of test automation

Key areas	Best practices	AL citation[a]	GL citation[a]	N
Test Automaton Strategy	Define an effective test automation strategy	Eva: [P1], [P2] Exp: [P3], [P4], [P5], [P6], [P7], [P8] Other: [P9], [P10], [P11], [P12], [P13],	Exp: [P27], [P28], [P29], [P30], [P31], [P32], [P33], [P34], [P35], [P36], [P37], [P38], [P39], [P40], [P41], [P42], [P43], [P44], [P45] Opi: [P46], [P47], [P48], [P49], [P50]	37
	Involve key stakeholders in strategy development	Exp: [P7], [P3] Other: [P11]	Exp: [P28], [P39], [P48]	6
	Adjust the strategy to the changes	—	Exp: [P36], [P28], [P51], [P52], [P33], Opi: [P50]	6
Resources	Provide enough resources	Eva: [P1] Exp: [P7], [P3], [P6], [P5] Other: [P14], [P10], [P11], [P12]	Exp: [P33], [P31], [P51], [P32], [P30], [P53], [P36]	16
Test organization	Acquire enough management support for test automation	Exp: [P5], [P7] Other: [P9], [P15], [P11], [P16]	Exp: [P54]	7
	Keep test professionals motivated about test automation	Other: [P16], [P11], [P15]	Exp: [P54], [P35]	5
	Have competent test professionals	Eva: [P1] Exp: [P3], [P17] Other: [P10], [P9], [P11], [P12]	Exp: [P28], [P30], [P35], [P51], [P36], [P32], [P55], [P37], [P53], [P40], [P56], [P43]; Opi: [P47], [P57], [P48]	22
	Promote collaboration	Other: [P16], [P51]	Exp: [P28], [P58], [P54], [P59], [P36], [P60], [P41], [P35], [P45] Opi: [P57]	12
Knowledge transfer	Share available test automation knowledge	Exp: [P6], [P3] Other: [P16]	Exp: [P34], [P61], [P30], [P35], [P32], [P62], [P63]	10
	Allow time for training and the learning curve	Other: [P15], [P10], [P9], [P11], [P16]	Exp: [P64], [P40], [P32], [P41], [P35], [P53], [P28] Opi: [P47]	13
Test tools	Select the right test tools	Eva: [P1], [P18], [P19], [P20], [P21], [P22] Exp: [P7], [P5], [P8], [P17], [P27] Other: [P11], [P9], [P12]	Exp: [P35], [P30], [P29], [P52], [P65], [P32], [P66], [P67], [P60], [P51], [P68], [P69] Opi: [P57], [P70]	28
	Properly use test tools	Exp: [P3], [P7] Other: [P15], [P23]	Exp: [P35], [P41], [P63]	7

(Continues)

TABLE 6 (Continued)

Key areas	Best practices	AL citation[a]	GL citation[a]	N
Test environment	Set up good test environments	Eva: [P20] Exp: [P3], [P7] Other: [P11], [P12]	Exp: [P28], [P34], [P32], [P66], [P36], [P51], [P28], [P64], [P65], [P40], [P35] Opi: [P48], [P56], [P71]	19
	Create high-quality test data	Other: [P13]	Exp: [P59], [P51], [P29], [P36], [P32], [P72] Opinion: [P57]	8
Test automation requirements	Define test automation requirements	Exp: [P6] Other: [P12]	Exp: [P32], [P34], [P36], [P45], [P69] Other: [P56], [P38]	9
	Have control over changes of test automation requirements	Exp: [P7] Other: [P24], [P12]	Exp: [P30], [P32], [P45]	6
Test design	Develop high-quality test scripts	Exp: [P7], [P5], [P3] Opi: [P70] Other: [P24], [P23], [P11], [P13]	Exp: [P28], [P32], [P73], [P52], [P36], [P55], [P35], [P74], [P34], [P75], [P63], [P76], [P44]	21
	Arrange testware in good architecture	Exp: [P7], [P5] Other: [P11], [P12], [P13]	Exp: [P28], [P61], [P29], [P77], [P76], [P73], [P74], [P17], [P64]	14
Test execution	Prioritize automated tests for execution	Exp: [P7]	Exp: [P36], [P61], [P64], [P56], [P60], [P41], [P32], [P72]	9
	Automate pre-processing and post-processing	Exp: [P7]	Exp: [P32]	2
Verdicts	Automate test oracles	—	Exp: [P32], [P69]	2
	Analyse test automation efficiently and effectively	Exp: [P7], [P3], [P5]; Other: [P11]	Exp: [P32], [P28], [P64], [P36], [P29], [P31], [P78]	11
	Report useful test automation results to key stakeholders	Exp: [P7] Other: [P25], [P12]	Exp: [P28], [P36] Opi: [P71]	6
Measurements	Use the right test automation metrics	Exp: [P7]; Other: [P12]	Exp: [P79], [P32], [P66], [P28], [P78], [P36] Opi: [P50], [P77]	10
SUT	Design the SUT for automated testability	Eva: [P18] Exp: [P5] Other: [P11], [P1], [P14], [P12]	Exp: [P73], [P29], [P66], [P42], [P60] Opi: [P56]	12
Technology	Adopt new technologies	—	Exp: [P51], [P69], [P68] Opi: [P80]	4

[a]The categories Eva, Exp, Opi and other have been introduced in Section 3.5.4.

4.2 | RQ2—The advice on how to conduct proposed best practices

Using cited sources (Table 6), we further explain 26 test automation best practices and present the advice given on how to conduct them.

4.2.1 | Test automation strategy related practices

Define an effective test automation strategy

37 sources (Table 6) suggested defining an effective test automation strategy to set action plans to conduct test automation within an organization. Academic studies [P1][P2], which respectively surveyed 33 and 72 industry professionals, found that the substantial improvement of test automation maturity can be achieved by effective strategic planning. As the guidance in the literature, many prior studies advised what main topics an effective test automation strategy should cover. Table 7 summarizes these main topics. The detailed analysis on how to define these main topics in strategy development was given in several test maturity models (proposed by authorized organizations). TMap [P3] advised the main steps to define test automation goals: consider expected outcomes, transfer expected outcomes into goals, and link the goals with organizational policies and business needs. It also advised the standard risk analysis process (from identify

TABLE 7 Test automation strategy

Main topic	Description	AL citation[a]	GL citation[a]
Goals	Define the specific goals of test automation for a short or/and long period of time, e.g., increase the product quality and user satisfaction, support frequent releases, reduce testing time, enhance the testing scale, minimize human efforts, accelerate feedback loop.	Exp: [P3], [P7]	Exp: [P28], [P34], [P36], [P39], [P38], [P45] Opi: [P46]
Test scope	Make decisions on which parts of the test scope shall be automated to what degree and to which test level	Exp: [P3], [P7]	Exp: [P30], [P9], [P36], [P11], [P33], [P38], [P32], [P28], [P45]
Risks	Predict and track risks associated with test automation	Exp: [P3], [P5] Other: [P11]	Exp: [P29], [P28], [P45] Opi: [P46]
Resources	Consider what resources are required and available for test automation	Exp: [P3], [P7]	Exp: [P32], [P35], [P34], [P28], [P39] Opi: [P47]
Costs & benefits	Analyse the benefits of test automation over its costs	Exp: [P9], [P10] Other:[P7], [P5]	Exp: [P32], [P37], [P30], [P32], [P39] Opi: [P38], [P48]
Effort	Estimate how many efforts will be spent on test automation based on the defined strategy	Exp: [P3], [P7]	Exp: [P32] Opi: [P47], [P38]

[a]The categories Eva, Exp, Opi and other have been introduced in Section 3.5.4.

risks, analyse the impact of risks, to define controlling measures) and resource allocation process (from identify demand test automation resources, assign resources and re-assign as necessary, to track the usage of resources) for test automation projects. STBox 3.0 [P28] advised several approaches to identify test automation scope, for example, based on prioritized features, stakeholder expectations, the ease of automation of test cases, and clients' feedback. It also illustrated the example approach to estimate test automation effort based on team members' performance. TestSPICE 3.0 [P36] introduced an approach of analysing the costs and benefits of multiple executions of automated tests.

Involve key stakeholders in strategy development

6 sources (Table 6) noted the importance of involving key stakeholders (e.g., testers, developers, managers and others who can affect or be affected by test automation) in strategy development and translate their expectations into meaningful decisions. Based on experience studies [P48][P39][P7], without key stakeholders' involvement, an organization may fail to identify important test automation needs, disappoint stakeholders, and miss out targets to make inputs. Referring to how to involve key stakeholders in strategy development, test maturity models TMap [P3] and STBox 3.0 [P28] suggested the formal approach—managers lead in working with key stakeholders to formally discuss the main topics on a test automation strategy—for the general software development contexts. Several practitioners [P48][P39] viewed that, based on their experience, in agile contexts, it can be done in an informal approach, which emphasizes quick decisions and 'agility in doing' (reactive planning) in responding to changing needs of key stakeholders. With such an informal approach, the main topics on a test automation strategy can be aware of at any time and informal discussion among key stakeholders will occur when necessary [P48][P39]. Yet, we did not found evaluation studies that validate or compare the effect of formal approach and informal approach in real practices.

Adjust the strategy to the changes

6 sources (Table 6) suggested adjusting the strategy to the changes, as they argued that change is the one true constant in test automation and it can occur in diverse forms, for example, the technology, test scope, or business scope changes, test environment or SUT updates, the use of new test approaches, and staff turnover. Practitioners [P52][P33] reported that, in their past projects, having a test automation strategy adjusted against the changes helped them to rethink their test automation needs and set up stepwise maturity improvements. As the guidance, test maturity model STBox 3.0 [P28] advised the main steps for adjusting the strategy against changes: review the strategy around main topics it includes, make changes with key stakeholders, and communicate the changes within an organization. The detailed analysis for each step can be found in STBox 3.0 [P28].

4.2.2 | Resources related practices

Provide enough resource

16 sources (Table 6) proposed this best practice. Based on an industrial survey [P51] that explored the state of software testing practices with around 2000 test professionals in the world, the major obstacle for improving test automation

TABLE 8 Resources

Type	Resources	AL citation[a]	GL citation[a]
Test environment	Software, hardware, networks, cloud, operating systems, test data, supporting tools to set up a test environment.	Exp: [P5], [P7] Other: [P51], [P68]	Exp: [P30], [P53], [P28]
Test tools	Test tools to support and perform test automation activities.	Exp: [P6], [P7] Other: [P51], [P68]	Exp: [P28]
Human resources	Skilled people, e.g., tool experts, test managers, testers, consultants, test automation experts.	Eva: [P1] Exp: [P5], [P7] Other: [P51]	Exp: [P28], [P30], [P33]
Costs	Development and upkeep costs of test automation. Direct costs: resource purchasing, software licensing, training, cloud and network services, and people hiring. Indirect costs: efforts spent on test automation.	Other: [P11], [P14], [P51]	Exp: [P32], [P31]

[a]The categories Eva, Exp, Opi and other have been introduced in Section 3.5.4.

maturity is resource shortage. An academic study [P1] that surveyed 33 industry professionals found that provide enough resources is a key success factor for improving test automation maturity. Many experience studies [P31][P7][P5] reported that, in strategy development, identifying demand resources and rationally allocating available resources for usage can avoid the risk of resource shortage to some extent. This links to 'Define en effective test automation strategy' related practices presented in Section 4.2.1. To support in identifying demand test automation resources, the literature introduced different types of test automation resources, see Table 8. Some practitioners [P33][P30] claimed that, based on their experience, using test management tools with resource management functions is helpful for rational resource allocation. Test maturity model TPI [P6] declared that management skills of managers also can affect the situation of resource allocation. This links to 'Have competent test professional' related practices presented in Section 4.2.3.

4.2.3 | Test organization related practices

Acquire enough management support for test automation

7 sources (Table 6) argued that, adopting test automation can bring work pattern, technology and organizational culture changes, while introducing such changes need enough management support within organizations. The academic study [P5] presented a case study of establishing automated regression testing in two projects at ABB (a pioneering technology company)—automated regression testing almost failed in the middle of these two projects, since inadequate management support was obtained to involve core team members and get resource provision. However, we did not identify the advice on how to acquire management support for test automation from cited sources (Table 6) of this best practice.

Keep test professionals motivated about test automation

5 sources (Table 6) claimed that the extent of motivation determines how much effort test professionals would spend on test automation and the use rate of test tools; and therefore, they suggest keeping test professionals motivated about test automation. As described by Fewster and Graham [P7], who had more than 20-year test automation experience, *the best automation tool in the world will not help test efforts, if your team resists using it*. Based on academic studies [P16] [P15] that surveyed the state of software testing practices in different organizations, having unmotivated test professionals may bring negative outcomes, for example, more turnover, less productivity, low test efficiency, narrow test automation scope, and dissatisfaction to test automation outcomes. However, we did not identify the advice on how to motivate test professionals about test automation from cited sources (Table 6) of this best practice.

Have competent test professionals

As reported in 22 sources (Table 6), test professionals need the competencies (a collection of knowledge, skills and abilities) to perform test automation tasks [P3][P47]. The academic study [P1] on surveying 33 industrial practitioners found that having competent test professionals is key to improve test automation maturity. Based on a SLR on the impediments of test automation maturity [P11], in terms of lacking competencies of test professionals, there is a high risk for test automation failures, low testing quality, and costs and efforts overdraft. The literature advised what competencies test professionals should have in test automation practices. Table 9 summarizes these competencies by role type: managerial role and technical role. Based on cited sources in Table 9, in a practice context, a test professional can have both managerial role and technical role. To develop test professionals' competencies, test maturity models TMap [P3] and TestSPICE 3.0 [P36] suggested organizing training programmes, sharing the expertise and learning from experience and practice. These practices are related to knowledge transfer related practices in Section 4.2.4. Experience studies [P35]

TABLE 9 Competencies of test professionals

	Test automation related competencies	AL citation[a]	GL citation[a]
Managerial role	Ability to develop, adjust and execute the test automation strategy within planning and budget	Exp: [P3], [P51] Other: [P11]	—
	Ability to coordinate test automation activities and stakeholders	Exp: [P3] Other: [P11]	—
	Knowledge and skills for test automation maturity improvement	—	Exp: [P51], [P28], [P32], [P53]
	Knowledge of architectures and tools for system development	Exp: [P3] Other: [P11]	Exp: [P35], [P51]
Technical role	Coding and scripting abilities	Exp: [P3] Other: [P11]	Exp: [P32], [P30], [P51], [P53], [P51] Opi: [P47]
	Expertise with test tools and the capability to use them	Other: [P11]	Exp: [P51], [P28], [P35]
	Knowledge about the SUT	Other: [P11]	Exp: [P53], [P51] Opi: [P47]
	Skills in creating and interpreting the test basis (e.g., requirements, test results, functional and technical design)	—	Exp: [P37], [P51], [P53]
	Ability to automate tests, support and maintain test artefacts, prepare and maintain test environment and test data, and analyse test automation results.	Exp: [P3] Other: [P11]	Exp: [P37], [P53], [P51]

[a]The categories Eva, Exp, Opi and other have been introduced in Section 3.5.4.

TABLE 10 Knowledge sharing

	Resources	AL citation[a]	GL citation[a]
Knowledge type	Test scripts, test cases, testing techniques, testing approaches, test tool expertise, test automation requirements, test automation metrics, test automation expertise, development guidelines, system requirements, test reports, best practices and lesson learned.	Exp: [P3], [P6]	Exp: [P30], [P32], [P35], [P34]
Place to share	Wikis, annotations, content management systems, internal/external open source repository, group discussion forums, instant messaging tools, networking platforms, training sessions.	Exp: [P3], [P61], [P30], [P35] Other: [P16]	Exp: [P63]

[a]The categories Eva, Exp, Opi and other have been introduced in Section 3.5.4.

[P5][P44] have confirmed the increased competencies of test professionals after organizing training programmes. Besides, TMap [P3] also advised organizations to hire new experienced individuals, for example, test automation experts, tool specialists or consultants with required competencies. The positive effect of hiring new experienced individuals on accelerating competence development has been noticed by practitioners [P35][P17] in past test automation practices.

Promote collaboration

12 sources (Table 6) declaimed that, test automation is a team effort that needs many individuals working together to reach the common goals, so that promoting collaboration in test automation practices is suggested. Based on the World Quality report 2017–2018 [P51], good collaboration can ensure that test automation efforts are distributed evenly, the competency of everyone is fully used, and the roles and responsibilities of individuals are clearly defined. The experience of practitioners [P58][P54] showed that, by collaborating more, people can better share knowledge and learn from others. However, from cited 12 sources (Table 6), we did not identify the advice on how to promote collaboration in test automation practices.

4.2.4 | Knowledge transfer-related practices

Share available test automation knowledge

10 sources (Table 6) suggested sharing available test automation knowledge to reduce duplicated efforts, encourage sharing and learning from others and collect knowledge for further reference and innovations. Based on academic studies [P16][P24] that surveyed different organizations about the state of their software testing practices, the results of not

sharing test automation knowledge are the drain of knowledge, decreased innovation, reduced efficiency and increased test automation costs. To provide guidance on sharing test automation knowledge, prior studies introduced different types of shareable test automation knowledge and also advised the general places to share it, as shown in Table 10. The details on how to share these types of test automation knowledge through advised places were not identified from cited sources in Table 10.

Allow time for training and learning curve

13 sources (Table 6) suggested organizing training to develop the competencies of test professionals in test automation practices. What competencies test professionals should have are presented in Table 9 (Section 4.2.3). As reported in Ranorex's technical report [P64], based on the experience of many test automation consultants, training provides both organizations and individuals as a whole with benefits: Organizations offer consistent test automation knowledge and development guidelines to individuals through training; by taking training, individuals are able to develop the required competencies and thus increase the performance in the current role. Many experience studies [P40][P41][P35][P53] reported that, the learning curve to develop competencies is different from individual to individual in practice—if the learning time needed for test automation is underestimated or reduce, individuals may suffer the difficulty to complete the steep learning curve and then become less productive and motivated for test automation. However, from cited sources in Table 6, we did not identify the advice on how to organize training and estimate/improve individuals' learning curves in test automation practices.

4.2.5 | Test tools related practices

Select the right test tools

28 sources (Table 6) recommended selecting the right test tools. According to the recent World Quality Reports [P68][P69] (published by Micro Focus, Capgemini and Sogeti), test tools available in the market vary in different functions, price, learning curve, configuration settings, attributes (e.g., usability, extendability and maintainability), programming

TABLE 11 Test tool selection criteria

Criterion	Description	AL citation[a]	GL citation[a]
Function	Consider which testing functions are currently required, e.g., unit testing, performance testing, functional testing, acceptance testing, GUI testing, non-functional testing.	Eva: [P21], [P19] Exp: [P7]	Exp: [P30], [P27]
Feature	Identify which features of a test tool are currently required, e.g., can be integrated with continuous integration server or/and bug tracking systems, uses scriptless test automation, provides test design patterns, supports test execution and logging, has a real-time dashboard and simple reporting.	Eva: [P19], [P21]	Exp: [P29], [P52], [P27]
Usability	Assess whether a test tool is easy to use, its training time is short, and its learning curve is low.	Eva: [P19], [P21]	Exp: [P27], [P18], [P7]
Maintainability	Investigate how easy a test tool can be maintained in the current organization	Eva: [P19], [P21] Exp: [P7]	Exp: [P30]
Compatibility	Consider the compatibility with the SUT and the current system architecture.	Eva: [P19]	Exp: [P30], [P35], [P29]
Costs & benefits	Analyse whether the potential benefits of buying a test tool would outweigh the costs.	Eva: [P19]	Exp: [P35], [P57], [P21]
Environmental constraints	Explore the environmental constraints that affect the use of a test tool, e.g., a test tool is specific to particular hardware, software, SUT, operating systems, and programming languages supported.	Eva: [P18], [P21], [P19] Exp: [P7]	Exp: [P30], [P32] Opi: [P57]
Organizational constrains	Understand organizational constraints that affect the use of a test tool, e.g., available resources, the budget, organizational policy, the innovation roadmap, and the development strategy of an organization.	Eva: [P21], [P19] Exp: [P7]	-
Licensing	Examine the licensing type of a test tool, e.g., open source or commercial licensing.	Eva: [P18], [P21], [P19]	Exp: [P30], [P18]
Vendor support	Check what support the tool vendor provides, e.g., availability of training, user community, warranty, installation, upgrades, help services.	Eva: [P18], [P21], [P19]	Exp: [P32], [P66], [P21]

[a]The categories Eva, Exp, Opi and other have been introduced in Section 3.5.4.

language and future development trends. Several scholars [P1] [P19] [P21] have surveyed test professionals from different organizations and evaluated that, the right test tools refer to 'test tools that can best fit the current test automation needs of an organization'. To guide organizations to select the right test tools, the literature has proposed test tool selection criteria and we conclude that in Table 11. The content validity of each criterion in Table 11 has been evaluated by a recent academic study [P19] that surveyed 89 test professionals in the industry. Compared to Raulamo-Jurvanen et al. [P19], academic studies [P18] [P21] that surveyed fewer number of test professionals also evaluated the content validity of some criteria. Yet, the conflict view was presented in an experience study [P17] that reports the 10-year experience of test automation maturity improvement in a small size DevOps team. It described that, selecting the right test tools based on pre-defined criteria is less useful than selecting each test tool with an experimentation mindset, because new needs that offend pre-defined criteria may emerge at any time.

Properly use test tools

7 sources (Table 6) indicated that the carefully selected test tools may not deliver the promised outcomes, if they are not properly used. The specific example was illustrated in Lee et al.'s academic study [P15] that surveyed the industry about the usage of test tools in practice. Their study results showed that, in sampled 73 organizations, defect management tools were not functioning as expected, since the use of these tools was limited to basic functions (such as defect reporting, recording and tracking) but seldom for more complex functions (like defect prediction and prevention). As reported by many practitioners [P35] [P41] [P63], based on their experience, properly using test tools requires sufficient test automation knowledge and tool expertise. This can be linked to 'Have competent test professionals' and 'Allow for training and learning curve' related practices presented in Sections 4.2.3 and 4.2.4, respectively.

4.2.6 | Test environment related practices

Set up good test environments

19 sources (Table 6) suggested setting up good test environments to improve test automation maturity. As defined by the software industry and research community, test environment refers to a context that consists of software, hardware, devices, network, storage, test data, servers and other facilities to execute automated tests [P51][P68][P69]. Based on many experience studies [P7][P40][P28][P64], poor test environments may cause the lack of supporting facilities,

TABLE 12 Characteristics of the good test environment

	Characteristics	AL citation[a]	GL citation[a]
Effectiveness	Close to the corresponding product environment	—	Exp: [P34], [P40], [P30], [P46]
Availability	Available all the times to run automated tests	Other: [P11]	Exp: [P46], [P81], [P66], [P51]
Stability	There is little chance for outage	Other: [P11], [P9]	Opi: [P47]
Agility	Easy to be configured/re-configured	Other: [P11]	Exp: [P40], [P32]
Consistency	Run in an identical state for the same tests in different test phases	—	Exp: [P32], [P35]
Reproducibility	Can quickly recover from the points of failures	Exp: [P7]	Exp: [P30], [P40]
Maintainability	Easy to maintain and update	—	Exp: [P51], [P28]

[a]The categories Eva, Exp, Opi and other have been introduced in Section 3.5.4.

TABLE 13 Approaches to set up/improve test environments

Approaches	Description	AL citation[a]	GL citation[a]
Production environment understanding	Comprehend the corresponding production environment	—	Exp: [P28], [P30] Opi: [P46]
Requirement analysis	Analyse test automation requirements for configurations of test environments	—	Exp: [P66], [P35], [P28]
User analysis	Understand the demand and usage of test teams	—	Exp: [P32]
Environment testing	Test new test environments before actual usage	Exp: [P7] Other: [P11]	—
Test environment automation	Automate test environment configuration activities (e.g., environment design, provision, deployment and operation) and the relevant supporting activities (e.g., planning, maintenance and self-healing)	—	Exp: [P32], [P51]

[a]The categories Eva, Exp, Opi and other have been introduced in Section 3.5.4.

unstable tests, test execution failures, and thus the loss of testing time and costs in test automation practices. To help the organizations to assess test environments, the literature attempted to characterize the good test environment. Table 12 collects the good test environment's characteristics from different sources. Besides, several studies advised approaches to set up good test environments and improve existing test environments, as summarized in Table 13. Technical reports [P32][P51] (from software organizations) mentioned that test environment automation has many benefits: increase the agility to configure/re-configure test environments, ease to defect the conflicts of configured artefacts, reduce the operation costs, and avoid human errors. They recommended test environment automation for test automation on a large scale, in a continuous integration context, and has multiple versions of test environments. Other approaches (presented in Table 12) were recommended by cited sources for the general test automation context.

Create high-quality test data

Referring to cited 8 sources (Table 6), test data quality affects test automation maturity. Based on the experience of practitioners [P59][P51][P29], low-quality test data can delivery wrong results, increase rework efforts and waste testing time in test automation. To support organizations to assess the quality of test data, the literature defined the main characteristics of high-quality test data, see Table 14. Furthermore, prior experience studies (in the GL) presented several approaches to ensure/improve test data quality, while researchers' advice in the AL was not identified. Many experience studies advised setting up the guidelines for test data generation and management within an organization. Table 15 collects what guidelines these experience studies advised to set up for. Experience studies [P59][P51] reported that, using test automation frameworks, which have built-in standard guidelines for test data generation and management and are comprised of supporting practices and tools, can help to create high-quality test data. The experience study [P59] also suggested that manually reviewing the delivered test data for quality is also helpful.

4.2.7 | Test automation requirements related practices

Define test automation requirements

9 sources (Table 6) suggested defining test automation requirements to state expected test automation features and functionalities. Based on technical reports [P45][P32] from software organizations, if test automation requirements are not defined, test automation may lose the right coverage and depth, consume development resources and have low performance. Test maturity models TMap [P3] and TestSPICE 3.0 [P36] suggested that, both test automation requirements and manual testing requirements are software testing requirements. They advised the example steps to define software

TABLE 14 High-quality test data

Characteristic	Description	AL citation[a]	GL citation[a]
Completeness	Contain all required information	—	Exp: [P28], [P36]
Accuracy	Store the accurate information that reflects a real situation	—	Exp: [P28], [P36], [P51]
Consistency	The piece of information does not contradict another piece of information	Exp: [P7]	Exp: [P32]
Usage	Easy to access and use	Exp: [P7]	Exp: [P36], [P51]
Maintenance	Easy to maintain and make changes	Exp: [P7]	Exp: [P36], [P73], [P59], [P51]
Reuse	Easy to reuse	Exp: [P7]	Exp: [P36], [P32]
Privacy	Compliance with data policies	—	Exp: [P59], [P28], [P51]

[a]The categories Eva, Exp, Opi and other have been introduced in Section 3.5.4.

TABLE 15 Test data generation and management guidelines

Guideline	GL citation[a]
What test data are required	Exp: [P28], [P36]
How to generate test data	Exp: [P59], [P29]
How to use, manage, and maintain test data	Exp: [P36], [P72]
When and how to update test data	Exp: [P59], [P36]
How to deal with legacy test data	Exp: [P59]
What data policies should follow	Exp: [P59], [P28], [P51]

[a]The categories Eva, Exp, Opi and other have been introduced in Section 3.5.4.

testing requirements according to system requirements, test levels, stakeholder expectations and product risks. The recent World Quality Reports [P69][P68] mentioned that test automation requirements are different from manual testing requirements. They noted that, in the current industry, many organizations are using machine learning techniques to automatically extract functional requirements from existing test cases (which supply inputs to test execution and outputs in response to inputs), but details on how they do that were not introduced.

Have control over changes of test automation requirements

6 sources (Table 6) mentioned that test automation requirements may change throughout a testing process, for example, when the major changes are made on the SUT, stakeholders alter expectations, business requirements change, and test automation goals shift, and hence, they recommended organizations to have control over changes of test automation requirements. Based on the experience of practitioners [P30][P7], if test automation requirements change at the early stage, communicating the changes with stakeholders and giving enough time to reset test automation may be easy; however, if the changes occur at the late stage, it would need lots of rework efforts. The advice on how to control changes of test automation requirements was given in the study [P7], which proposed test automation guidelines according to the authors' decades of test automation experience and field studies of case organizations. Based on this study [P7], the actions that can be taken to control the requirement changes include: analysing risks of requirement changes at the initial phase, preparing flexible test cases to automate, automating the tests which remain unchanged first, and producing flexible and changeable test scripts in nature. The detailed analysis on each action is described in the same study [P7]. We found that, the action 'producing flexible and changeable test scripts' links to 'Develop high-quality test scripts' related practices in Section 4.2.8.

4.2.8 | Test design related practices

Develop high-quality test scripts

Developing high-quality test scripts (written in test code) is important to mature test automation, as presented in 21 sources (see Table 6). Based on an SLR [P11] on the impediments of test automation maturity, low-quality test scripts may make automated tests to be prone to errors (e.g., security vulnerabilities, unreliable results and coding standard violations) and difficult to run, maintain and reuse. To help the organizations to assess test script quality, prior studies characterized the high-quality test scripts. Table 16 collects the characteristics of high-quality test scripts from different sources. Several experience studies reported approaches to create or improve test scripts; Table 17 presents these approaches.

TABLE 16 Characteristics of high-quality test scripts

	Characteristic	AL citation[a]	GL citation[a]
Size	Written in small size	Exp: [P7]	Exp: [P32]
Function	Have the clear goal(s) to perform a single task	—	Exp: [P34], [P36]
Structure	Have the clear and simple structure that are easy to understand	Exp: [P7]	Exp: [P73], [P36], [P56]
Maintenance	Easy to maintain and make changes	Exp: [P7]	Exp: [P36], [P73], [P56]
Reuse	Easy to be reused by several test cases	Exp: [P7]	Exp: [P36], [P34]

[a]The categories Eva, Exp, Opi and other have been introduced in Section 3.5.4.

TABLE 17 Approaches to create and improve test script quality

Approach	Description	AL citation[a]	GL citation[a]
Formal check	Check whether the given test scripts properly run on the SUT with respect to test cases and business requirements	–	Exp: [P36]
Use scripting techniques	e.g., structure scripting, shared scripts, data-driven testing, keyword-driven testing. Each technique has its drawbacks and strengths. Two or more techniques can be used together in practice.	Exp: [P7]	Exp: [P56]
Statistic code analysis	Analyse test codes to eliminate coding issues	Exp: [P7]	Exp: [P74]
Development guidelines	Define development guidelines to train testers to comply with the same coding style, standards, and conventions	—	Exp: [P35]
Peer review	Do a peer review on test codes written by others to eliminate coding issues	—	Exp: [P36], [P35]

[a]The categories Eva, Exp, Opi and other have been introduced in Section 3.5.4.

As reported in cited sources [P7][P36][P28] (Table 17), on the basis of some approaches, relevant supporting tools have been developed: scripting technique based tools, statistic code analysis tools and test automation frameworks (provide standard development guidelines). Formal check and Peer review need human judgment [P35][P36].

Arrange testware in good architecture

Arranging testware (e.g., test cases, test scripts, expected results, actual results, test logs, test data and test reports) in good architecture was recommended by 14 sources (Table 6). Many practitioners [P29][P77][P73][P74] reported that, based on their experience, the poor architecture makes it difficult to track, reuse, and maintain the growing testware in extensive test automation, and the associated costs would be high. Based on white papers [P61][P64] from software organizations, some test tools have the default testware architecture that can be simply implemented. Recent experience studies [P77][P29] reported that, in a long-term perspective, it is better to design own testware architecture to fit an organization's own test automation context. Some practitioners learned from past experience and noted key issues to be considered in the design of testware architecture. Table 18 summarized these key issues.

4.2.9 | Test execution related practices

Prioritize automated tests for execution

9 sources suggested prioritizing the important automated tests for execution to enable early fault detection, fast feedback and decreased costs for ensuring product quality. The literature presented two prioritization methods used in test automation practices, see Table 19. Based on cited sources [P60][P32][P30][P36], requirement based prioritization normally prioritize customer requirements at first according to some factors, for example, customer demand, business value and fault impact of requirements. Based on cited sources [P7][P36], Regression based prioritization is goal-oriented—it first sets the goals (e.g., early fault detection, product quality improvement and fewer costs) and then using test tools and algorithms to prioritize automated tests based on the relevant metrics. TestSPICE 3.0 [P36] presented case studies for both types of prioritization.

Automate pre-processing and post-processing

Two sources [P7] and [P32] suggested automating pre-processing (that sets prerequisites before the test execution) and post-processing (that deals with the aftermath of test execution) in test automation practices. They argued that pre-processing and post-processing related tasks are tedious and manually perform them consuming testing efforts. The advice on how to automate pre- and post-processing was identified in the study [P7], an early source that proposed test automation guidelines based on authors' decades of test automation experience and field studies of case organizations. The study from Fewster and Graham [P7] collected a set of pre-processing and post-processing related tasks that should

TABLE 18 Key issues to be considered in the design of testware architecture

Issue	Description	AL citation[a]	GL citation[a]
Access	How to let testers and test tools to easily locate and access required testware	—	Exp: [P77], [P29]
Scale	How to support the arrangement of testware that grow in size	—	Exp: [P77], [P7]
Reuse	How to maximize reuse of testware	Exp: [P5]	—
Multiple versions	How to manage the multiple versions of testware	Exp: [P5]	—
Differentiation	How to manage and distinguish the different types of testware	Exp: [P22]	—
Independence	If there is the need to test the SUT in multiple environments/hardware platforms, how to store and manage the independent testware of each environment/platform	Exp: [P22]	Exp: [P76]

[a]The categories Eva, Exp, Opi and other have been introduced in Section 3.5.4.

TABLE 19 Test prioritization

Type	Description	AL citation[a]	GL citation[a]
Requirement based prioritization	Prioritize automated tests based on the priority of customer requirements.	—	Exp: [P60], [P32], [P30], [P36]
Regression based prioritization	Prioritize automated tests for regression testing, when the changes are made on the SUT/automated tests, after fixing the bugs, or finishing the updates of test environments.	Exp: [P7]	Exp: [P36]

[a]The categories Eva, Exp, Opi and other have been introduced in Section 3.5.4.

TABLE 20 Pre-processing and post-processing tasks [P7]

	Tasks
Pre-processing	Create files, test data, databases; prepare test environments; format test data and test scripts; install libraries and dependencies.
Post-processing	Delete files, databases, datasets; reorganize test results; format outcomes.

TABLE 21 Approaches to analyse test automation results

Approaches	Description	AL citation[a]	GL citation[a]
Interpret and classify test automation results	Analyse failed tests to find the root reason for failure and classify the results to prevent potential incidents.	—	Exp: [P36], [P34], [P32]
More than 'pass' or 'fail'	Store and review the artefacts (logs, screenshots, comparisons, or video recordings of test runs, and others generated from executing automated tests) to get complement information for debugging and fixing issues.	Exp: [P5]	Exp: [P36], [P41], [P32]
Notifications	Set the notifications (on test execution tools) to alarm the failures of critical automated tests, so that the priority can be given to analyse and solve the failures of critical automated tests when receiving the notifications.	Exp: [P7]	Exp: [P31]
Tool support	Use test tools that can give a clear overview of each step of the test flow so that failures can be quickly identified.	—	Exp: [P78]
Smoke tests	Run smoke tests on automated test suites incrementally to expose reasons for failures.	—	Opi: [P80]
Big picture	In addition to analyse a single test run results, it is essential to combine test automation results collected from different sources (e.g., across multiple test tools, test runs, configurations, integration builds, and milestones) into a big picture view of outcomes.	Exp: [P28], [P7]	—
Keep history	Store test automation results for a period of time to enable progress tracking, regression identification, and flaky tests identification.	Exp: [P32], [P7]	—

[a]The categories Eva, Exp, Opi and other have been introduced in Section 3.5.4.

be automated, see Table 20. As advised by the study [P7], these pre- and post-processing tasks can be implemented in shared scripts and command flies (e.g., command procedure, shell script and batch file) so that they can be automatically performed by test execution tools. The examples scripts and command files were given in the study [P7].

4.2.10 | Verdicts related practices

Automate test oracles

Two sources [P32][P9] studied the effect of test oracle on test automation maturity. The source [P9] is an SLR. The source [P32] is a technical report from STAT COE centre that supports US government testing and evaluation programmes. Based on these two sources, test oracle is a mechanism to verify the correctness of outputs of automated test cases concerning expect results. Both sources suggested that test oracle automation can reduce human efforts in verifying the outputs from executing test cases and thus increase the testing scale in a short test cycle. Yet, these two sources did not include the advice on how to conduct test oracle automation in practice.

Analyse test automation results efficiently and effectively

11 sources (Table 6) noted that efficiently and effectively analyse test automation results is important to mature test automation. As reported by practitioners [P29][P31], in their past practices, when the time spent on analysing test automation results exceeded the time saved from executing automated tests, test automation slowed the development work and it was not worth the cost. Experience studies [P29][P64] reported that ineffective test automation results harm software quality. Many sources (include experience and opinion studies) advised the approaches to efficiently and effectively analyse test automation results, as summarized in Table 21. The study [P7] presented examples to implement the suggested approaches (notifications, big picture, and keep history) on test execution tools.

Report useful test automation results to key stakeholders

As mentioned in 6 sources (Table 6), once test runs were executed, the next thing is test automation result reporting, which is the approach to make test automation results visible to stakeholders and inform them about their work. Based on test maturity models TestSPICE 3.0 [P36] and STBox 3.0 [P28], effective reporting can enable the track of test progress, make the tests reliable, and increase the quality of automated tests. They advised organizations to automatically report test automation results to key stakeholders using test tools. However, practitioners [P7] claimed that, based on their decades' test automation experience, not all reports generated by test tools are useful for all stakeholders, and thus, it is important to have a selection and only send the required reports to the right stakeholders. The study [P7] illustrated examples of what types of reports different stakeholders would prefer: decision makers might be interested in quality reports and progress reports to observe how test automation supports software development; developers would like to get bug reports, crash reports and error reports in order to get feedback about their development work and identify causes and fixes; testers would also be interested in crash reports and error reports to monitor the test automation status and performance.

4.2.11 | Measurement related practices

Use the right test automation metrics

10 sources (Table 6) recommended using the right test automation metrics to quantitatively measure test automation performance and maturity improvement actions. Based on many experience studies [P7][P32][P66][P79][P66], the right test automation metrics refers to the ones 'that are objective, measurable, and meaningful to serve the test automation goals within an organization'; the wrong metrics might waste efforts and obtain irrelevant/misleading data to measure test automation performance and maturity improvement actions. Several sources (Table 22) presented a collection of example test automation metrics. The study [P7] observed each of these test automation metrics in several cases of software organizations and presented the details on the usage of each metric in practice. Based on the experience of several practitioners [P66][P79], some tools (e.g., SonarQube and GitLab) also provide the collection of test automation metrics and metric data can be collected and recorded automatically. Experience studies [P7][P79][P78] suggested organizations define and customize test automation metrics based on their own needs, but how to do that was not introduced in these studies.

4.2.12 | SUT-related practices

Design the SUT for automated testability

12 sources (Table 6) mentioned that it is important to design the SUT for automated testability. An academic case study [P18] on test automation with visual GUI tools found that the SUT testability affects test automation maturity. As reported by several experience studies [P29][P60][P73], many SUTs might not be testable for test automation, for example, the SUT that changes often, is not stable, is difficult to write test scripts on that, or has low availability and running speed. As concluded in an SLR [P11] on the impediments of test automation maturity, the SUT with low automated testability might need an extensive workload to develop test automation, test automation functions may fail on some

TABLE 22 Test automation metrics

Measurement	Example metrics	AL citation[a]	GL citation[a]
Test automation coverage	Lines of code tested by test automation; Increased coverage of expected operational paths and use cases; Percentage of test coverage achieved by automated tests.	Exp: [P7]	Exp: [P79] Opi: [P50]
Test efficiency	Average time to design and execute automated tests; Average time to run test automation sequence; Average time to provide feedback to development works; Percentage of automated tests passed, skipped, and failed w.r.t. total number of automated tests planned to run.	Exp: [P7]	Exp: [P79] Opi: [P50]
Test effectiveness	Percentage of useful test automation results; Defect detection percentage = defects found by automated tests / total known defects; Defect fix percentage = defects fixed before release / all defects found.	Exp: [P7]	Exp: [P79]
Product quality	Number of defects in production; Satisfaction rate of users	Exp: [P7]	Exp: [P79]
Maintenance efforts	Number of defects in production; Reuse rate of test automation artefacts; The average elapsed time to update automated tests.	Exp: [P7]	Exp: [P79]

[a]The categories Eva, Exp, Opi and other have been introduced in Section 3.5.4.

parts of the SUT, and thus software development productivity suffers. However, from cited sources in Table 6, we did not identify the advice on how to design the SUT for automated testability.

4.2.13 | Technology related practices

Adopt new technologies

4 sources (Table 6) suggested adopting new technologies in test automation practices. Based on World Quality Reports [P51][P68][P69] that annually surveyed global quality and testing practices and analysed the trends, in the area of test automation, there was the evolution when new technologies were commoditized in the industry: In the early years, test automation was only used to do simple regression tests with reply and record tools; these days, test automation technologies have become more complex and that are being used to carry out different levels of tests (unit test, integration test and performance test) and support diverse testing activities (e.g., test design, test execution, measurements, test environment provision, defect prediction and test management); in the future, advanced technologies have the promise to build more intelligent test automation that is more effective and efficient to test more. As reported by industrial experts in these reports, based on their estimation, software organizations staying with old technologies may lose the competencies in the future; thus, they suggested software organizations follow test automation technical trends in the industry and watch for opportunities for action. However, from cited sources (Table 6) of this best practice, the advice on how to adopt new technologies within an organization was not identified.

5 | DISCUSSIONS

We summarize and discuss the study finding in Section 5.1, explore the implications to research in Section 5.2 and the implications to practice in 5.3, and examine threats to validity in Section 5.4.

5.1 | Summary and discussion of study findings

This MLR has the objective to survey and synthesize the guidelines given in the literature for test automation maturity improvement. To solve the objective, from a large pool of sources, we selected and reviewed 81 primary studies (including 55 GL and 26 AL sources) on this topic. Many of our primary studies are experience reports ($n = 34$) that proposed test automation heuristics and guidelines (Figure 4). Next, we answer our research questions and discuss study findings on them.

5.1.1 | RQ1. Which test automation best practices are given in the literature?

As an answer to RQ1, from the literature, we extracted 26 test automation best practices and grouped them into 13 key areas, as presented in Table 6. As described in Section 4.1, there are only six best practices whose positive effect on maturity improvement have been validated by academic evaluation studies using formal empirical methods. Future academic evaluation studies are needed to validate the effect of other best practices on test automation maturity improvement.

Our previous work [14] is the only prior academic study that also proposed test automation best practices. Our previous work collected test automation best practices from 18 test maturity models, which are used to guide test automation practices in the industry. Six technical-related best practices proposed in this MLR were not presented in our previous work: Set up good test environments, Create high-quality test data, Develop high-quality test scripts, Automate test oracles, Analyse test automation results efficiently and effectively, and Adopt new technologies. This denotes that the recent AL and GL (reviewed in this MLR) proposed test automation best practices that were not covered in test maturity models (reviewed in our previous work). Once the positive effect of these six technical-related best practices on test automation maturity has been validated with cross-site empirical evidence, there is a need to update test maturity models that miss these best practices. Additionally, one management related best practice 'Establishing a test organization (like a test team or a department) to assemble people to perform test automation tasks' was presented in some test maturity models (reviewed in our previous work) but was not identified from the current AL and GL (reviewed in this MLR). The possible reason could be the wide adoption of continuous integration drives the change. In continuous integration contexts, development and test automation are integrated, and thus, in many cases, developers perform both development and test automation tasks [24]. Yet, further studies are needed to explore the details.

As discussed in Table 1 (Section 2.3), to provide the guidelines for test automation maturity improvement, Rodrigues et al. [31] proposed a taxonomy of test automation success factors and Wiklund et al. [13] proposed a taxonomy of

TABLE 23 Rodrigues et al.'s [31] success factors vs. our key areas of best practices

	Test Automation strategy	Resources	Test organization	Knowledge transfer	Test tools	Test environment	Test automation requirements	Test design	Test execution	Verdicts	Measurements	SUT	Technology
Feasibility assessment	Yes												
Testability Level of the SUT												Yes	
Resource availability		Yes											
Resource Reusability		Yes											
Well Defined Test Process	Yes												
Scalability								Yes					
Maintainability								Yes					
Automation Tool Acquisition					Yes								
Quality Control											Yes		
Dedicated and Skilled Team			Yes										
Automation Planning/Strategy	Yes		Yes										

maturity impediments. We found that, our taxonomy of test automation best practices in this MLR and their taxonomy of test automation success factors and maturity impediments can be linked together. Table 23 shows which key areas of our best practices can be linked to each success factor in Rodrigues et al.'s study. Table 24 shows which key areas of our best practices can be linked to each category of maturity impediments in Wiklund et al.'s study. Compared to their work, our MLR covers more test automation key areas. Rodrigues et al.'s study did not present success factors in knowledge transfer, test environment, test automation requirements, test execution, Verdicts, and Technology key areas, see Table 23. Wiklund et al.'s study did not identify maturity impediments in test execution, measurement, and technology key areas, see Table 24. Additionally, referring to the gap analysis in Table 1, their studies did not present how to address success factors and impediments (they proposed) in practice. Our MLR complements their work. Our best practices with the collected advice (in RQ2) on how to conduct them can be used to address relevant success factors and maturity impediments in their work.

5.1.2 | RQ2. What advice was given in the literature about how to conduct proposed test automation best practices?

From the literature, we identified many pieces of advice (in forms of implementation or improvement approaches, actions, technical techniques, concepts, experience-based heuristics) on how to conduct proposed test automation best practices in RQ1. We have several observations as described below:

We can see from Section 4.2, most pieces of advice on how to conduct proposed best practices were identified from experience studies and their effectiveness needs to be evaluated with cross-site empirical evidence using formal empirical methods. Only some pieces of advice on how to 'Select the right test tools' (Section 4.2.5) have been evaluated by academic studies with cross-site empirical evidence: to support organizations to select right test tools, the literature has

TABLE 24 Wiklund et al.'s [13] categories of maturity impediments vs. Our key areas of best practices

	Test Automation strategy	Resources	Test organization	Knowledge transfer	Test tools	Test environment	Test automation requirements	Test design	Test execution	Verdicts	Measurements	SUT	Technology
Behavioural effects	Yes		Yes										
Business and planning	Yes	Yes	Yes										
Skills			Yes	Yes									
Test system					Yes	Yes	Yes	Yes		Yes			
SUT												Yes	

defined the concept of 'the right test tools' and proposed selection criteria, and the content validity of the concept and selection criteria have been evaluated by academic studies that surveyed cross-site test professionals.

Several pieces of advice on how to conduct some best practices are conflicting. Qualitative studies are needed to explain the conflicts. First, as the advice for 'Involving key stakeholders in strategy development' (Section 4.2.1), some test maturity models (proposed by authorized organizations) suggested the formal approach for general software development context—managers lead in working with key stakeholders to formally discuss the main topics on a test automation strategy. Meanwhile, some practitioners viewed that, based on their experience, in agile contexts, it can be done in an informal approach—the main topics on a test automation strategy can be aware of at any time and informal discussion among key stakeholders will occur when necessary. Second, as the advice for 'Select the right test tools' (Section 4.2.5), many scholars suggested selecting test tools against pre-defined selection criteria, while some practitioners viewed that—based on their experience—selecting test tools against pre-defined criteria is less useful than selecting each test tool with an experimentation mindset. Third, as the advice for 'Define test automation requirements' (Section 4.2.7), some test maturity models suggested that both test automation requirements and manual testing requirements are software testing requirements, while recent industry reports mentioned that test automation requirements are different from manual testing requirements.

The pieces of advice on how to conduct some best practices still need further qualitative analysis. First, as the advice for 'Share available test automation knowledge' (Section 4.2.4), prior studies discussed different types of shareable test automation knowledge and advised places to share it, but did not include details on how to share these types of shareable test automation knowledge through advised places. Second, as the advice for 'Define test automation requirements' (Section 4.2.7), recent industry reports mentioned that many organizations are using machine learning techniques to automatically extract functional requirements from existing test cases, but details on how they do that were not introduced. Third, as the advice for 'Use the right test automation metrics' (Section 4.2.11), the literature defined the concept of 'right test automation metrics' and illustrated a collection of example ones, but it did not mention how to define and customize test automation metrics based on own needs of an organization.

We did not identify the advice on how to conduct some management related best practices: 'Acquire enough management support for test automation' (Section 4.2.3), 'Keep test professionals motivated' (Section 4.2.3), 'Promote collaboration' (Section 4.2.3), 'Allow time for training and learning curve' (Section 4.2.4), 'Adopt new technology' (Section 4.2.13). However, we found that SE research has given advice for adopting similar management related best practices in SE contexts. For example, researchers [43] have advised the main steps to acquire management support for SE activities within an organization: engage key managers, express expected results, frame a project in the context of organization objectives, gather feedback, and share good news. The study [44] proposed a process to promote collaboration in SE, from bringing awareness of collaboration in the organizational culture to establishing real-time collaboration with tools, environments, and infrastructure to facilitate informal and formal communications. The study [45] observed that SE practitioners can be motivated by several motivators: increased salary, career promotion, technical growth, personal identity with tasks, autonomy, and working environment. Software process model CMMi [46] proposed the main activities of an organizational training programme: identify training needs, providing training, establishing and maintaining the training capability and records, assessing training effectiveness. Nemoto et al. [47] described four variables that should be considered in order to successfully adopt the new computer technology within

an organization: attributes of innovation, external environment, suppliers, organizational characteristics. As test automation is a component of SE, with some modifications, such pieces of advice from SE research can be immigrated into the test automation context, although evaluation and guidance studies are required to validate the effect.

The advice on how to conduct technical related best practices 'Design the SUT for automated testability' (Section 4.2.12) and 'automate test oracles' (Section 4.2.10) was not found from our primary studies. However, to our best knowledge, there exist technical related AL on these two practices. To extend the study, we applied the search string 'SUT testability automat*' and 'test oracle automat*' in Google Scholar searching for technical AL on these two practices. We reviewed relevant studies and found that much work remains to be done in order to support the industry to conduct these two practices. First, we observed that, the approaches to design automated testability are very diverse and highly dependent on the application domain of the SUT and test types. Some technical approaches have been proposed to design the SUT for automated testability for embedded systems, GUI test automation, and objective-oriented systems [48–50]. Yet, technical approaches to design automated testability of SUT for other application domains (e.g., mobile applications and software-defined systems) and test types (e.g. unit testing, integration testing, and performance testing) are still needed [48–51]. Second, we observed that technical techniques have been proposed to support test oracle automation. Barr et al. [52] has constructed a repository of 694 publications on test oracles and studied the existing technical techniques for test oracle automation. They found that the existing technical techniques (including modelling, specifications, contract-driven development, and metamorphic testing) are not sufficient, as the human intervention still need to provide the final source of test oracle information, for example, informal specifications, expectations, norms, and domain-specific information—as such, test oracle automation still needs relatively high costs, and thus, how to keep the benefits while reducing the costs is still a challenge. This calls for more technical studies on test oracle automation.

5.2 | Implications to research

This MLR narrows the gap between practice and research—the industry needs to improve test automation maturity improvement, while the research effort on surveying and synthesizing the guidelines given in the literature for improving test automation maturity is limited. This MLR spots research challenges and opportunities around four topics:

5.2.1 | Empirical studies

More empirical studies are acutely needed. In this MLR, we only identified 10 evaluation studies presenting empirical results, despite the industry relevance of this research topic. This MLR points out empirical study opportunities. First, there are only six test automation best practices whose positive effect on maturity improvement have been evaluated by academic studies using formal empirical methods, and thus, future academic studies are needed to evaluate the effect of other best practices on maturity improvement as well. Researchers can adopt such best practices in a given industrial context, for example, in a specific domain like embedded software systems, in small and medium size organizations, or in a continuous integration context, and then observe the effect. The relationships between different best practices and the intended effects also can be observed. Second, in this MLR, we collected many pieces of advice (from different sources) on how to conduct the proposed best practices, while most of them come from experience studies. Studies are needed to observe the consequence of following the collected pieces of advice to conduct the proposed best practices in different organizations and as such evaluate the effectiveness of these pieces of advice.

5.2.2 | Qualitative studies

26 test automation best practices in this MLR notice the important research topics for test automation research. Due to the scope of this MLR, we only provided an overview of the pieces of advice (proposed by prior researchers and practitioners) on how to conduct these best practices. Thus, we suggest more case studies to observe how these 26 best practices can be conducted against the collected pieces of advice. For example, case studies can be conducted on the adoption of the suggested approaches to set up/ improve test environments (Table 13), create and improve test script quality (Table 17), and analyse test automation results (Table 21) within different organizations. Additionally, referring to the observations on study results of RQ2 in Section 5.1.2, future studies are needed to explain the conflicts among the pieces of advice on how to conduct certain best practices ('Involving key stakeholders in strategy development', 'Select the right test tools', and 'Define test automation requirements'), and add further qualitative analysis to the pieces of advice on how to conduct certain best practices ('Share available test automation knowledge', 'Define test automation requirements', 'Use the right test automation metric').

5.2.3 | Technical studies

Referring to observations to study results of RQ2 (Section 5.1.2), technical studies are required to (1) develop approaches for designing automated testability of the SUT in different application domains and test types and (2) and advance test oracle automation techniques with least human efforts. Besides, as test automation is technology-driven and technology maturity is an important dimension of test automation maturity (see definition in Section 2.1), more technical studies are required to advance the next level of test automation maturity.

5.2.4 | Maturity models

Test maturity models are being used to guide test automation practices in the industry. However, referring to observations to study results of RQ1 (Section 5.1.1), this MLR identified six technical related best practices that were not found in test maturity models (reviewed in our previous work [14]): Set up good test environments, Create high-quality test data, Develop high-quality test scripts, Automate test oracles, Analyse test automation results efficiently and effectively, and Adopt novel technologies. Once the positive effect of these six technical-related best practices on improving test automation maturity has been evaluated, future work is needed to update test maturity models that miss these best practices. Additionally, many researchers denoted the need to develop test automation maturity models, since test maturity models focus more on manual testing than test automation [11,53,54]. Other researchers can use our study results of this MLR as a base to develop test automation maturity models. The application of the updated models or new models should be evaluated by research and practice.

5.3 | Implications to practice

Our MLR proposed 26 test automation best practices and collected the pieces of advice on how to conduct them. It has implications for organizations that are doing test automation or improving test automation maturity. To be more specific, by consulting our study results, organizations may find 26 test automation best practices to follow during the planning and the implementation of test automation. Additionally, they can also compare their existing test automation practices with these 26 best practices to assess the current state of their test automation practices. Our MLR identified the pieces of advice (from prior scholars and practitioners) on how to conduct the proposed 26 best practices. Referring to such pieces of advice around these best practices, organizations may get hints about the next steps for improving their test automation maturity.

5.4 | Threats to validity

We identified the main threats that may affect the validity of this MLR according to the guidelines from Petersen and Gencel [55]. Their guidelines provide a taxonomy of validity threats in SE research.

5.4.1 | Descriptive validity

Descriptive validity determines 'the extent to which the observations of a study are described objectively and accurately [55]'. In SLR studies, researchers ignore important data on primary studies may lead to a threat to descriptive validity. To control this threat, we defined review questions and specified what data should be extracted from primary studies. Accordingly, we coded all relevant findings from primary studies. During the coding process, the original sources were always reviewed to ensure the validity of codes.

5.4.2 | Theoretical validity

Theoretical validity refers to the extent to which a study reflects what it intends to reflect [55]. Some relevant studies were excluded. This may threaten the theoretical validity of this MLR. We restricted our search to English language sources, though some relevant studies have been published in other languages. Additionally, human decisions on search selection may imperil the theoretical validity. To control that, we ensured the source selection process involves two authors. Sources were finally selected according to the selection results of two authors, and final decisions were shown to other co-authors for the review.

5.4.3 | Interpretive validity

Interpretive validity focus on how conclusions are reasonably drawn on the given data in a study [55]. Researcher bias may introduce a threat to this type of validity. The interpretation of data may depend on the experience and thoughts of researchers of this paper. Though the interpretation of data was made by the first author, the conclusions were reviewed by other authors. The second and fourth authors are experienced in conducting MLRs and have published extensively in the field of software testing or SE. Besides, when presenting conclusions, examples were used in many places to provide more information.

5.4.4 | Generalizability

Generalizability concerns the extent to which the results of a study can be generalized [55]. In this MLR, we take care of how study results can be generalized to the industrial and academic contexts. Practitioners need to consider individual differences in their test automation practices when consulting study results of this MLR. Besides, SE is a rapidly evolving discipline and the generalizability of study results depends on the current context [55]. Hence, the threat lies in whether our research findings can be generalized in the field of SE after a couple of years later.

5.4.5 | Repeatability

Repeatability refers to the extent to which a study is repeatable [55]. To avoid threats to repeatability, we carried out this MLR by following 'the guidelines for including grey literature and conducting MLRs' from Garousi et al. [16]. The search strategy was defined and it was accordingly followed to conduct this MLR. Intermediary results of research stages were stored in spreadsheets and NVivo files. The whole research process was reported in detail.

6 | CONCLUSION

Given the popularity of test automation and the large investments that might be wasted with negative outcomes from being immature, software organizations must improve test automation maturity. This MLR set the study objective to survey and synthesize the guidelines given in the current literature for test automation maturity improvement. In solving the study objective, this MLR selected and reviewed 81 primary studies consisting of 26 AL and 55 GL sources on this topic. From primary studies, this MLR extracted 26 test automation best practices and collected many pieces of advice (in forms of implementation/improvement approaches, actions, technical techniques, concepts and experience-based opinions) on how to conduct these best practices.

This MLR has four main contributions. First, it proposed 26 test automation best practices to suggest steps for improving test automation maturity. Practitioners can consult these best practices to improve test automation maturity or build test automation from scratch, see details in Section 5.3. Second, it narrows the gap between practice and research—the industry needs to improve test automation maturity, while the current research lacks the synthesis on this topic. Third, this MLR provides a centralized knowledge base of existing guidelines given by prior scholars and practitioners for test automation maturity improvement. This centralized knowledge base can be used to get an overview of research advancements in this field, and frost future research topics that merit attention. Last, this MLR identified research challenge and opportunities around four topics: empirical studies to validate the study results of this MLR, qualitative studies to explain some details and conflicts, technical studies to advance test automation technology, and maturity model related studies, see Section 5.2 for details.

ACKNOWLEDGEMENTS

This work is supported by TESTOMAT Project (ITEA3 ID number 16032) funded by Business Finland (under grant decision ID 3192/31/2017), and the foundation of Tauno Tönning (project ID 20210086).

ORCID

Yuqing Wang https://orcid.org/0000-0003-0175-005X

REFERENCES

1. Capgemini, Sogeti, Microfocus. World quality report 2019–20. In Technical Report, Capgemini and Sogeti and Microfocus, 2019. https://www.capgemini.com/research/world-quality-report-2019/

2. Pressman RS. Software engineering: A practitioner's approach. Palgrave Macmillan, 2005.
3. Bubevski V. A novel approach to software quality risk management. Softw Test Verif Reliab. 2014;24(2):124–54.
4. Garousi V, Mäntylä MV. When and what to automate in software testing? A multi-vocal literature review. Inform Softw Technol. 2016;76: 92–117.
5. MarketsandMarkets Research. Automation testing market, 2019. https://www.marketsandmarkets.com/Market-Reports/automation-testing-market-113583451.html
6. ISTQB. Istqb glossary, 2021. https://glossary.istqb.org/app/en/search/
7. Wang Y, Mäntylä MV, Demeyer S, Wiklund K, Eldh S, Kairi T. Software test automation maturity: a survey of the state of the practice. In *Proceedings of the 15th International Conference on Software Technologies - Volume 1: ICSOFT*. INSTICC, SciTePress, 2020;27–38.
8. Garousi V, Pfahl D. When to automate software testing? A decision-support approach based on process simulation. J Softw Evol Process. 2016; 28(4):272–85.
9. ISTQB Worldwide software testing practices survey 2019–20. In Technical Report, ISTQB, 2020 https://www.istqb.org/references/surveys.html
10. ISTQB. Worldwide software testing practices survey 2017–18. In Technical Report, ISTQB, 2018. https://www.istqb.org/references/surveys.html
11. Wiklund K. Impediments for automated software test execution. *Ph.D. Thesis*, Mälardalen University, 2015.
12. Garousi V, Elberzhager F. Test automation: not just for test execution. IEEE Softw. 2017;34(2):90–6.
13. Wiklund K, Eldh S, Sundmark D, Lundqvist K. Impediments for software test automation: a systematic literature review. Softw Test Verif Reliab. 2017;27(8):e1639.
14. Wang Y, Mäntylä M, Eldh S, et al. A self-assessment instrument for assessing test automation maturity. In *Proceedings of the Evaluation and Assessment on Software Engineering*, ACM, 2019;145–54.
15. Ricca F, Stocco A. Web test automation: insights from the grey literature. In *International Conference on Current Trends in Theory and Practice of Informatics*, Springer, 2021;472–85.
16. Garousi V, Felderer M, Mäntylä MV. Guidelines for including grey literature and conducting multivocal literature reviews in software engineering. Inform Softw Technol. 2019;106:101–21.
17. Stevenson A. Oxford dictionary of English. OUP Oxford, 2010. https://books.google.fi/books?id=anecAQAAQBAJ
18. Paulk MC, Weber CV, Curtis B, Chrissis MB. The capability maturity model: guidelines for improving the software process, vol. 441. Addison-wesley: Reading, MA, 1995.
19. Tarhan A, Turetken O, Reijers HA. Business process maturity models: a systematic literature review. Inform Softw Technol. 2016;75:122–34.
20. Mettler T. Maturity assessment models: a design science research approach. Int J Soc Syst Sci (IJSSS). 2011;3(1/2):81–98.
21. Pocatilu P. Automated software testing process. Economy Inform. 2002;1:97–9.
22. Fewster M, Graham D. Software test automation (1st edn.) Addison-Wesley: Harlow [U.A.], 1999.
23. Ammann P, Offutt J. Introduction to software testing. Cambridge University Press, 2016.
24. Garousi V, Felderer M, Kuhrmann M, Herkiloğlu K, Eldh S. Exploring the industry's challenges in software testing: an empirical study. J Softw Evol Process; 32(8):e2251.
25. Koomen T, Broekman B, van der Aalst L, Vroon M. TMap next: for result-driven testing. Uitgeverij kleine Uil, 2013.
26. Kitchenham B, Brereton OP, Budgen D, Turner M, Bailey J, Linkman S. Systematic literature reviews in software engineering—a systematic literature review. Inform Softw Technol. 2009;51(1):7–15.
27. Garousi V, Felderer M, Mäntylä MV. The need for multivocal literature reviews in software engineering: complementing systematic literature reviews with grey literature. In *Proceedings of the 20th International Conference on Evaluation and Assessment in Software Engineering*, 2016;1–6.
28. Garousi V, Felderer M, Hacaloğlu T. Software test maturity assessment and test process improvement: a multivocal literature review. Inform Softw Technol. 2017;85:16–42.
29. Myrbakken H, Colomo-Palacios R. Devsecops: a multivocal literature review. In *International Conference on Software Process Improvement and Capability Determination*, Springer, 2017;17–29.
30. Rafi DM, Moses KRK, Petersen K, Mäntylä MV. Benefits and limitations of automated software testing: systematic literature review and practitioner survey. In *Proceedings of the 7th International Workshop on Automation of Software Test*, IEEE Press, 2012;36–42.
31. Rodrigues A, Dias-Neto A. Relevance and impact of critical factors of success in software test automation lifecycle: a survey. In *Proceedings of the 1st Brazilian Symposium on Systematic and Automated Software Testing*, ACM, 2016;6.
32. Brereton P, Kitchenham BA, Budgen D, Turner M, Khalil M. Lessons from applying the systematic literature review process within the software engineering domain. J Syst Softw. 2007;80(4):571–83.
33. Ćurković M, Košec A. Bubble effect: including internet search engines in systematic reviews introduces selection bias and impedes scientific reproducibility. BMC Med Res Methodol. 2018;18(1):1–3.
34. Wang X, Bendersky M, Metzler D, Najork M. Learning to rank with selection bias in personal search. In *Proceedings of the 39th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2016;115–24.
35. Mowshowitz A, Kawaguchi A. Measuring search engine bias. Inform Process Manag. 2005;41(5):1193–205.
36. Yasin A, Fatima R, Wen L, Afzal W, Azhar M, Torkar R. On using grey literature and Google scholar in systematic literature reviews in software engineering. IEEE Access. 2020;8:36226–43.
37. Zhou Y, Zhang H, Huang X, Yang S, Babar MA, Tang H. Quality assessment of systematic reviews in software engineering: a tertiary study. In *Proceedings of the 19th International Conference on Evaluation and Assessment in Software Engineering*, ACM, 2015;14.
38. Kitchenham B. Procedures for performing systematic reviews. Keele, UK, Keele Univ. 2004;33(2004):1–26.
39. Petersen K, Vakkalanka S, Kuzniarz L. Guidelines for conducting systematic mapping studies in software engineering: an update. Inform Softw Technol. 2015;64:1–8.
40. Keele S, et al. Guidelines for performing systematic literature reviews in software engineering. In Technical Report, Citeseer, 2007.
41. Petersen K, Feldt R, Mujtaba S, Mattsson M. Systematic mapping studies in software engineering. In *12th International Conference on Evaluation and Assessment in Software Engineering (EASE) 12*, 2008;1–0.
42. Cruzes DS, Dyba T. Recommended steps for thematic synthesis in software engineering. In *2011 International Symposium on Empirical Software Engineering and Measurement*, IEEE, 2011;275–84.
43. Ragu-Nathan BS, Apigian CH, Ragu-Nathan TS, Tu Q. A path analytic study of the effect of top management support for information systems performance. Omega. 2004;32(6):459–71.

44. Whitehead J. Collaboration in software engineering: a roadmap. In *Future of Software Engineering (FOSE'07)*, IEEE: Minneapolis, MN, USA, 2007;214–25.
45. Verner JM, Babar MA, Cerpa N, Hall T, Beecham S. Factors that motivate software engineering teams: a four country empirical study. J Syst Softw. 2014;92:115–27.
46. Team CP. Capability maturity model® integration (CMMI SM), version 1.1. CMMI for systems engineering, software engineering, integrated product and process development, and supplier sourcing (CMMI-SE/SW/IPPD/SS, V1. 1). 2002;2.
47. Nemoto MCMO, Vasconcellos EPG, Nelson R. The adoption of new technology: conceptual model and application. J Technol Manag Innov. 2010;5(4):95–107.
48. Mariani L, Pezzè M, Riganelli O, Santoro M. Automatic testing of GUI-based applications. Softw Test Verif Reliab. 2014;24(5):341–66.
49. Kanstren T. A study on design for testability in component-based embedded software. In *2008 Sixth International Conference on Software Engineering Research, Management and Applications*, IEEE: Prague, Czech Republic, 2008;31–8.
50. Kansomkeat S, Rivepiboon W. An analysis technique to increase testability of object-oriented components. Softw Test Verif Reliab. 2008;18(4): 193–219.
51. Yoo S, Harman M. Regression testing minimization, selection and prioritization: a survey. Softw Test Verif Reliab. 2012;22(2):67–120.
52. Harman M, McMinn P, Shahbaz M, Yoo S. A comprehensive survey of trends in oracles for software testing. In Tech. Rep. CS-13-01, University of Sheffield, 2013.
53. Eldh S, Andersson K, Ermedahl A, Wiklund K. Towards a test automation improvement model (TAIM). In *2014 IEEE Seventh International Conference on Software Testing, Verification and Validation Workshops*, IEEE: Cleveland, OH, USA, 2014;337–42.
54. Furtado APCC, Meira S, Gomes M. Towards a maturity model in software testing automation. In *The Ninth International Conference on Software Engineering Advances*, 2014;282–5.
55. Petersen K, Gencel C. Worldviews, research methods, and their relationship to validity in empirical software engineering research. In *2013 Joint Conference of the 23rd International Workshop on Software Measurement and the 8th International Conference on Software Process and Product Measurement*, IEEE, 2013;81–9.

How to cite this article: Wang Y, Mäntylä MV, Liu Z, Markkula J, Raulamo-jurvanen P. Improving test automation maturity: A multivocal literature review. Softw Test Verif Reliab. 2022;32:e1804. https://doi.org/10.1002/stvr.1804

APPENDIX A: PRIMARY STUDIES

P1–P26 are academic literature sources. P27–P81 are grey literature sources

REFERENCES

P1. Rodrigues A & Dias-Neto A Relevance and impact of critical factors of success in software test automation lifecycle: a survey. In Proceedings of the 1st Brazilian Symposium on Systematic and Automated Software Testing, ACM, 2016;6.
P2. Garousi V, Felderer M, Kuhrmann M, Herkiloğlu K, Eldh S. Exploring the industry's challenges in software testing: an empirical study. J Softw Evol Process. 2020;32(8):e2251.
P3. Koomen T, Broekman B, van der Aalst L, Vroon M. TMap next: for result-driven testing. Uitgeverij kleine Uil, 2013.
P4. Furtado APCC, Meira S, Gomes M. Towards a maturity model in software testing automation. In *The Ninth International Conference on Software Engineering Advances*, 2014;282–5.
P5. Persson C, Yilmazturk N. Establishment of automated regression testing at ABB: industrial experience report on 'avoiding the pitfalls'. In *Proceedings of the 19th IEEE International Conference on Automated Software Engineering*, IEEE Computer Society, 2004;112–21.
P6. Koomen T, Pol M. Test process improvement: a step-by-step guide to structured testing. Addison-Wesley, 1999.
P7. Fewster M, Graham D. Software test automation (1st edn.) Addison-Wesley: Harlow [U.A.], 1999.
P8. Khan AZ, Iftikhar S, Bokhair RH, Khan ZI. Issues/challenges of automated software testing: a case study. Pakistan J Comput Inform Syst. 2018;3(2):61–75.
P9. Garousi V, Mäntylä MV. When and what to automate in software testing? A multi-vocal literature review. Inform Softw Technol. 2016;76: 92–117.
P10. Garousi V, Felderer M, Hacaloğlu T. Software test maturity assessment and test process improvement: a multivocal literature review. Inform Softw Technol. 2017;85:16–42.
P11. Wiklund K, Eldh S, Sundmark D, Lundqvist K. Impediments for software test automation: a systematic literature review. Softw Test Verif Reliab. 2017;27(8):e1639.
P12. Wang Y, Mäntylä M, Demeyer S, Wiklund K, Eldh S & Kairi T Software test automation maturity: a survey of the state of the practice. In Proceedings of the 15th International Conference on Software Technologies - ICSOFT,. INSTICC, SciTePress, 2020; 27–38.
P13. Ricca F, Stocco A. Web test automation: insights from the grey literature. In *International Conference on Current Trends in Theory and Practice of Informatics*, Springer, 2021;472–85.
P14. Kasurinen J, Taipale O, Smolander K. Software test automation in practice: empirical observations. Adv Softw Eng. 2010;2010(4):1–3.
P15. Lee J, Kang S, Lee D. Survey on software testing practices. IET Softw. 2012;6(3):275–82.

P16. Deak A & Stålhane T Organization of testing activities in norwegian software companies. In 2013 IEEE Sixth International Conference on Software Testing, Verification and Validation Workshops (ICSTW), IEEE, 2013;102–107.
P17. Wang Y, Pyhäjärvi M & Mäntylä MV Test automation process improvement in a DevOps team:experience report. In 2020 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW), IEEE, 2020;314–321.
P18. Borjesson E, Feldt R. Automated system testing using visual GUI testing tools: a comparative study in industry. In *2012 IEEE Fifth International Conference on Software Testing, Verification and Validation*, IEEE, 2012;350–9.
P19. Raulamo-Jurvanen P, Hosio S, Mäntylä MV. Practitioner evaluations on software testing tools. Proceedings of the Evaluation and Assessment on Software Engineering, 2019;57–66.
P20. Kasoju A, Petersen K, Mäntylä MV. Analyzing an automotive testing process with evidence-based software engineering. Inform Softw Technol. 2013;55(7):1237–59.
P21. Illes T, Herrmann A, Paech B, Rückert J. Criteria for software testing tool evaluation. A task oriented view. In *Proceedings of the 3rd World Congress for Software Quality*, vol. 2, 2005;213–22.
P22. Haugset B & Hanssen GK The home ground of automated acceptance testing: mature use of fitnesse. In 2011 Agile Conference, IEEE, 2011;97–106.
P23. Wiklund K, Eldh S, Sundmark D & Lundqvist K Technical debt in test automation. In 2012 IEEE Fifth International Conference on Software Testing, Verification and Validation, IEEE, 2012;887–892.
P24. Hushalini S, Randunu RPAA, Maddumahewa RM, Manawadu CD. Software test automation in practice: empirical study from sri lanka. Compusoft. 2014;3(11):1232.
P25. Garousi V, Elberzhager F. Test automation: not just for test execution. IEEE Softw. 2017;34(2):90–6.
P26. Rafi DM, Moses KRK, Petersen K & Mäntylä MV Benefits and limitations of automated software testing: systematic literature review and practitioner survey. In Proceedings of the 7th International Workshop on Automation of Software Test, IEEE Press, 2012;36–42.
P27. Guru99. Automation testing tutorial: what is automated testing? https://www.guru99.com/automation-testing.html, Last accessed: October 2021.
P28. CTG Europe. Stbox 3.0 generic software testing process framework, 2009. https://www.ctg.com/sites/default/files/users/user7/STBox3.0.pdf
P29. Tellicherry T. 7 best practices in test automation, 2017. https://www.asktester.com/15-building-blocks-test-automation-strategy/, Last accessed: October 2021.
P30. Yang L. Factors to consider when implementing automated software testing. https://apps.dtic.mil/dtic/tr/fulltext/u2/1022569.pdf, Last accessed: October 2021.
P31. Chilott S. The testing maturity model, 2016. https://www.ir.com/blog/the-testing-maturity-model, Last accessed: October 2021.
P32. Simpson J & Wisnowski J Automated software testing implementation guide. In Technical Report, 2017.
P33. Schmidt M. How mature is your automation strategy? 2016. https://devops.com/mature-automation-strategy/, Last accessed: October 2021.
P34. Eriksson U. The 15 essential building blocks of a test automation strategy, 2017. https://www.asktester.com/15-building-blocks-test-automation-strategy/ Last accessed: October 2021.
P35. Dustin E. Lessons in test automation, 1999. https://www.stickyminds.com/better-software-magazine/lessons-test-automation, Last accessed: October 2021.
P36. TestSPICE SIG Testspice—process assessment model. In Technical Report, 2014.
P37. Graham D. Journey to test automation maturity, 2004. https://www.stickyminds.com/presentation/journey-test-automation-maturity-0, Last accessed: October 2021.
P38. Aspire Systems. Test automation—a reality check. https://ukalf.com/sites/default/files/Test%20Automation%20-%20A%20Reality%20Check.pdf, Last accessed: October 2021.
P39. Infosys. Creating business value with mature QA practices. In Technical Report, 2017. https://www.infosys.com/IT-services/validation-solutions/white-papers/Documents/creating-business-value.pdf
P40. Colantonio J. 145: automation testing maturity curve with Danny McKeown, Test Talks, 2017. https://www.joecolantonio.com/testtalks/145-automation-testing-maturity-curve-danny-mckeown/, Last accessed: October 2021.
P41. Weinberg U. Introducing automation to your organization: a strategic move, 2015. https://techbeacon.com/app-dev-testing/introducing-automation-your-organization-strategic-move, Last accessed: October 2021.
P42. Gryka M. When is a test case ready for test automation? 2017. https://dzone.com/articles/when-is-a-test-case-ready-for-test-automation-1, Last accessed: October 2021.
P43. Angie J. Measure your test automation maturity, 2020. https://dzone.com/articles/measure-your-test-automation-maturity, Last accessed: October 2021.
P44. Watson M. How to measure defect escape rate to keep bugs out of production, 2017. https://dzone.com/articles/how-to-measure-defect-escape-rate-to-keep-bugs-out, Last accessed: October 2021.
P45. Altexsoft. 11 ways to improve software testing through planning, work environment, automated testing, and reporting, 2021. https://www.altexsoft.com/blog/engineering/software-testing-qa-best-practices/, Last accessed: October 2021.
P46. Pavan R. Practical approach for improving agile testing maturity part 1, 2016. https://www.cigniti.com/blog/practical-approach-for-improving-agile-testing-maturity/, Last accessed: October 2021.
P47. Hagar J. Complex, critical software system testing: when automation is the only option, 2018. https://www.stickyminds.com/article/complex-critical-software-system-testing-when-automation-only-option, Last accessed: October 2021.
P48. Crombie M. Benefits of automated testing as an app matures, 2018. https://darkiceinteractive.com/articles/benefits-of-automated-testing-as-an-app-matures/, Last accessed: October 2021.
P49. Sharp-Paul A. Automation maturity model, 2013. https://www.upguard.com/blog/configuration-automation-maturity-model, Last accessed: October 2021.
P50. Kayla M. 5 key ways to report on automation project success. In Technical Report, 2019. https://www.performancemagazine.org/5-ways-report-automation-project/
P51. Capgemini, Sogeti & Microfocus. World quality report 2017–18, 2017 https://www.sogeti.com/explore/reports/world-quality-report-2017-2018/
P52. Yan S & Automated web application testing: Pm's guide. in Technical Report. Nov 1, 2017. https://www.scnsoft.com/software-testing/automated-web-app-testing-pm-guide, Last accessed: January 2022.

P53. Krause MH. Software: a maturity model for automated software testing, 1994. https://www.mddionline.com/software-maturity-model-automated-software-testing, Last accessed: October 2021.
P54. Pettichord B. Three keys to test automation, 2000. https://www.stickyminds.com/article/three-keys-test-automation, Last accessed: October 2021.
P55. Johnson DW. Is automated testing replacing the software tester? 2011. https://searchsoftwarequality.techtarget.com/tip/Is-automated-testing-replacing-the-software-tester, Last accessed: October 2021.
P56. Chernyak M. Evaluating the effectiveness of test automation, 2015. https://www.a1qa.com/blog/evaluating-the-effectiveness-of-test-automation/, Last accessed: October 2021.
P57. Helderson A. 5 tips to maximize your test automation ROI, 2017. https://www.utest.com/articles/5-tips-to-maximize-your-test-automation-roi, Last accessed: October 2021.
P58. Stackify. Software testing tips: 101 expert tips, tricks and strategies for better, faster testing and leveraging results for success, 2017. https://stackify.com/software-testing-tips/, Last accessed: October 2021.
P59. Jim A. Test data management maturity assessment, 2015. https://eyeonquality.com/test-data-management-maturity-assessment/, Last accessed: October 2021.
P60. PractiTest. How to coordinate your automated and manual testing. https://www.practitest.com/qa-learningcenter/best-practices/coordinate-automated-and-manual-testing/, Last accessed: October 2021.
P61. CenturyLink. Today's it demands new approaches in QA and test. In Technical Report, 2016. https://www.centurylink.com/asset/business/enterprise/white-paper/todays-it-demands-new-approaches-qa-test-whitepaper-WP160022.pdf
P62. Ch4n. Maturity model for test organization, 2007. https://blogs.msdn.microsoft.com/testing123/2007/03/21/maturity-model-for-test-organization/ Last accessed: October 2021.
P63. Ramanenka U. How 3 process patterns will change the way you do test automation, 2021. https://medium.com/bumble-tech/how-3-process-patterns-will-change-the-way-you-do-test-automation-277acc0d3171, Last accessed: October 2021.
P64. Ranorex 10 essential strategies for a successful automation project. In Technical Report, 2018 https://www.ranorex.com/resources/whitepaper/automation-project-strategies/
P65. Systeme Evolutif Limited Test organisation maturity questionnaire, 1998 https://cm.techwell.com/sites/default/files/articles/XML0161_0.pdf
P66. Skytap. Planning for long-term test automation success, 2016. https://www.asktester.com/15-building-blocks-test-automation-strategy/, Last accessed: October 2021.
P67. Atkins T. Cots tools vs. scripting languages for test automation, 2002. http://thinktesting.com/articles/cots-tools-vs-scripting-languages-for-test-automation/, Last accessed: October 2021.
P68. Capgemini, Sogeti, Microfocus. World quality report 2019–20. In Technical Report, 2019. https://www.capgemini.com/research/world-quality-report-2019/
P69. Capgemini, Sogeti, Microfocus. World quality report 2020–21, 2020. https://www.sogeti.com/explore/reports/world-quality-report-2020/
P70. Bose S. 10 test automation best practices to follow, 2020. https://www.browserstack.com/guide/10-test-automation-best-practices, Last accessed: October 2021.
P71. Edstrom J. How mature QA organizations can scale automation, 2017. https://magenic.com/thinking/how-mature-qa-organizations-can-scale-automation, Last accessed: October 2021.
P72. Thompson B. https://thinktesting.com/articles/value-of-test-automation/, 2016. Last accessed: October 2021.
P73. Noggle BJ. 6 ways to improve your agile automated testing process, 2016. https://www.getzephyr.com/insights/6-ways-improve-your-agile-automated-testing-process, Last accessed: October 2021.
P74. Will B. State-driven testing: an innovative approach to UI test automation, 2012. https://community.microfocus.com/t5/Silk-Test-Tips-Information/State-Driven-Testing-An-Innovative-Approach-to-UI-Test/ta-p/1758051, Last accessed: October 2021.
P75. Ghahrai A. Why would you want to automate a test? 2018. https://www.testingexcellence.com/why-would-you-want-to-automate-a-test/, Last accessed: October 2021.
P76. El-Deeb A. How to build test automation from scratch? A top-down approach, 2016. https://www.linkedin.com/pulse/how-build-test-automation-from-scratch-top-down-approach-el-deeb/, Last accessed: October 2021.
P77. Gupta Y. Measurement of quality of the testing and test automation process. https://www.softwaretestinggenius.com/measurement-of-quality-of-the-testing-and-test-automation-process/, Last accessed: October 2021.
P78. Wainstein L. Testing in a DevOps mindset, 2017. https://www.codementor.io/limormaayan/testing-in-a-devops-mindset-eb4ndxtcf, Last accessed: October 2021.
P79. Sealights. 11 test automation metrics and their pros & cons. https://www.sealights.io/test-metrics/11-test-automation-metrics-and-their-pros-cons/, Last accessed: October 2021.
P80. Young M. Achieve higher levels of maturity in testing automation, 2018. https://www.functionize.com/blog/achieve-higher-levels-of-maturity-in-testing-automation/, Last accessed: October 2021.
P81. Graham D. Test automation objectives, 2009. https://www.bcs.org/upload/pdf/aug09-automationobjectives.pdf, Last accessed: October 2021.